

ARCHITECTURE & DESIGN

HORIZON GRID

Architecture — Data Flow, AI, Providers, Deployment, and Security

Version: 0.2.4

Documentation prepared: 2026-08-21

PREPARED FOR EVALUATION

Table of Contents

Technical Architecture Overview	3
Data Flow	9
AI Architecture (Technical)	12
Provider Architecture	18
Database Architecture	23
Windows Deployment Architecture	26
Security Architecture	31
Linux Packaging Architecture	37

Technical Architecture Overview

HORIZON GRID is a self-hosted tool that looks up an **IOC** (Indicator of Compromise — e.g. an IP address, domain, URL, or file hash) against many third-party threat-intelligence sources at once, then uses an AI model to summarize and correlate the results. The backend's own self-description (`backend/app/main.py`) puts it this way:

"single-search IOC lookup across dozens of providers, correlated and summarized by a local Ollama model (or AWS Bedrock/Gemini/Anthropic)."

This section describes what the platform is built from at the container/process level, not the request-level data flow (covered elsewhere in this document) or the AI/provider internals (also covered elsewhere).

Container Inventory

`docker-compose.yml` defines seven backing services — `postgres` , `redis` , `neo4j` , `opensearch` , `backend` , `celery_worker` , and `celery_beat` — plus the `frontend` container, for eight containers running the full stack.

Container	Image / stack	Role	Implementation status
postgres	postgres:16-alpine	System of record: lookups, provider results, AI summaries, correlation edges, evidence, cases, users.	Actively used — the only datastore actually written to during a lookup.
redis	redis:7-alpine	Provider-result cache (logical DB /0), per-user rate limiter, Celery message broker (/1) and Celery result backend (/2).	Actively used.
neo4j	neo4j:5-community + APOC plugin	Config field and container exist to "mirror" correlation edges into a graph store, per two module docstrings.	Provisioned only — no driver instantiation, Cypher query, or read/write call exists anywhere in the backend code. Not used for anything today.
opensearch	opensearchproject/opensearch:2.17.0 (DISABLE_SECURITY_PLUGIN: "true")	Config field and container exist for an intended full-text/search index.	Provisioned only — no indexing or search client code exists in the backend. No search functionality is implemented against it.
backend	FastAPI, Python 3.12	Core API server: IOC-type detection, provider orchestration, AI summarization/correlation, evidence building, auth, case management.	Actively used — the primary application process.
celery_worker	Celery 5.4 (same backend codebase)	Executes background jobs.	Actively used, but for exactly one job (see below).
celery_beat	Celery 5.4 beat scheduler	Triggers scheduled jobs on a timer.	Actively used, one scheduled job only.
frontend	Next.js 14.2.15	Web UI.	Actively used.

A second compose file, `docker-compose.prod.yml`, is layered on top of `docker-compose.yml` for production use (this is what the Windows installer runs): it strips the development bind-mounts and switches the `backend` / `frontend` containers to their production start commands rather than dev/hot-reload ones.

Neo4j and OpenSearch, specifically: both are real, running containers in every deployment of this stack, and both have a corresponding settings field in the backend config (`neo4j_uri` / `neo4j_user` / `neo4j_password` , `opensearch_url`). Neither is wired into any actual code path. The correlation graph that the product presents to users lives entirely in Postgres's `correlation_edges` table today. Any description of a "Neo4j-backed graph" or "OpenSearch-backed search" elsewhere should be read as an architecturally-provisioned but not-yet-integrated capability, not a working feature.

Backend

The backend is a **FastAPI** application (`backend/app/main.py`) running on **Python 3.12**. Its notable stack choices:

- **SQLAlchemy 2.0**, async, with `asyncpg` as the Postgres driver.
- **Alembic** for schema migrations (`backend/alembic/`).
- **Celery 5.4** for the one background job described below.
- **structlog** and **prometheus-fastapi-instrumentator** for logging and metrics.

The backend is also where the 16 third-party data-source connectors ("providers"), the AI summarization/correlation logic, and the authentication/RBAC (role-based access control) system live — each of those is detailed in its own section of this document.

Frontend

The frontend is a **Next.js 14.2.15** application using **React 18.3.1** and **TypeScript**, styled with **Tailwind**, with **Zustand** for client-side state, **Recharts** for charts, **react-force-graph-2d** for the relationship/correlation graph visualization, and **Radix UI** for primitive UI components (all per `frontend/package.json`).

`package.json` declares `vitest` as the test runner (`"test": "vitest run"`), but a repository-wide search found zero `*.test.*` / `*.spec.*` files under `frontend/` — the tooling is configured but no frontend automated tests currently exist. (See the Testing and Quality Assurance appendix for the full picture, including the backend's test suites.)

Executive Dashboard and Deterministic Scoring Engine

A new **Executive Dashboard** (`GET /api/v1/dashboard/kpis` , backed by `app/core/dashboard.py::get_kpis()`) gives an at-a-glance operational view of the whole platform: real-time KPI tiles for active investigations, the count of critical/high-risk IOCs, open case count and open-critical case count (reported as two separate numbers, not collapsed into one), average threat score, provider health percentage, and AI analysis success rate. Every one of these is a live SQL aggregate computed against Postgres at request time — none is hardcoded, and none is cached client-side across page loads. A companion endpoint, `GET /api/v1/dashboard/executive-summary` (`app/ai/dashboard_summary.py`), layers an AI-generated 2–4 sentence narrative on top of the exact same KPI values, grounded exclusively in those numbers as given facts. If the AI backend is unreachable, or its structured response fails validation twice in a row, the endpoint falls back to a deterministic, template-generated sentence built directly from the same real KPI numbers — never an error message, and never fabricated data. This is disclosed to the user directly on the Dashboard itself: the Executive Summary card carries an "AI-generated" badge when the narrative came from the model, or a "Template fallback" badge when it didn't, so the source of the text is never ambiguous.

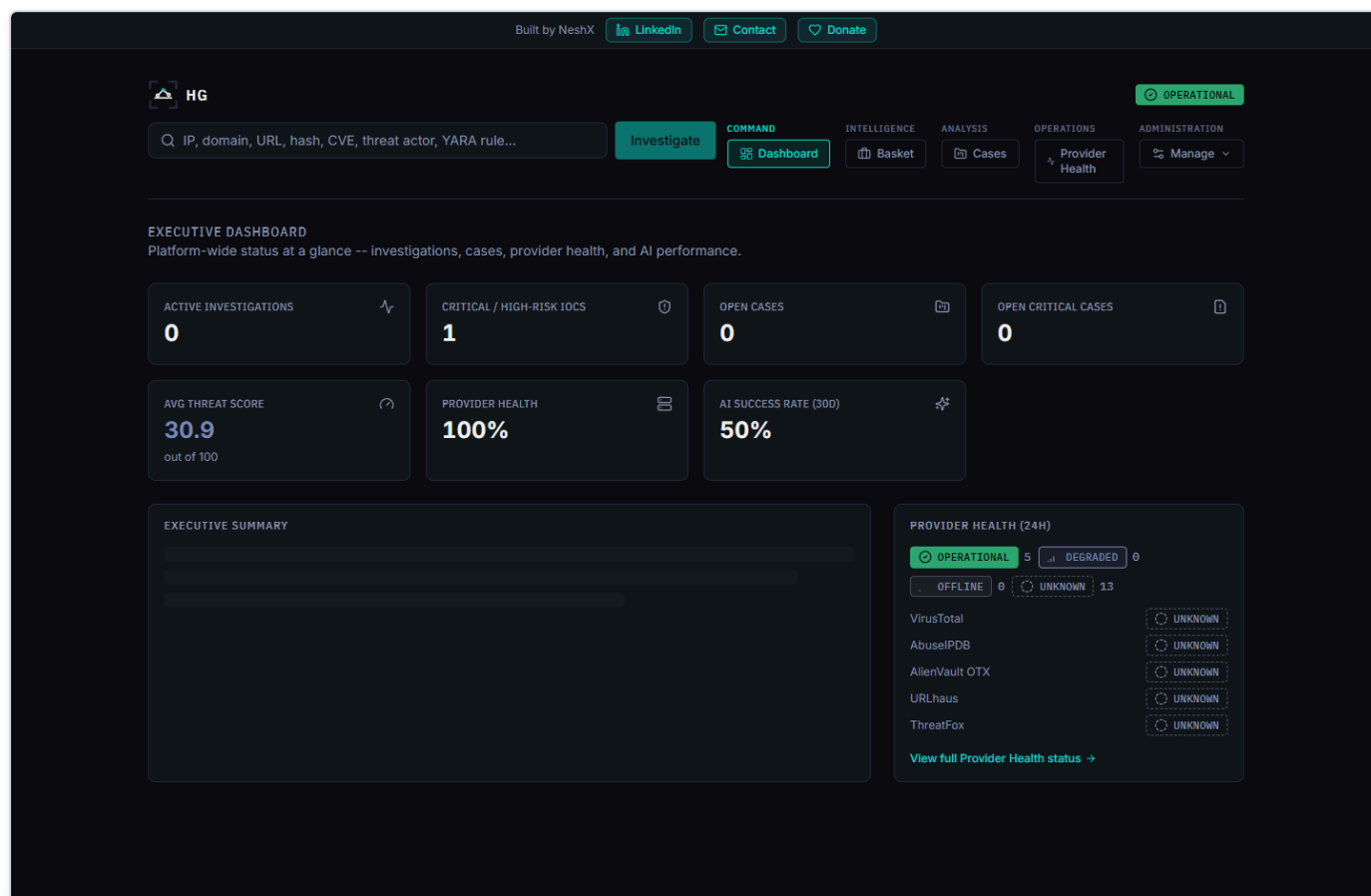


Figure 1 — The Executive Dashboard shows real KPI tiles, an AI-generated (or template-fallback) executive summary, and a provider-health summary widget -- every number sourced live, never hardcoded.

Underlying both the Dashboard's "average threat score" tile and every individual investigation's risk score is a new **deterministic (non-AI) scoring engine** (`app/scoring/engine.py`). Historically, `overall_risk_score`, `confidence_score`, `malicious_probability`, and the final verdict were 100% AI-generated in one call — prompt wording alone cannot reliably stop a model from overriding a number it was merely asked not to touch. The scoring engine instead computes all four values *before* any AI call runs, purely from already-fetched evidence: a cross-provider verdict-consensus component (weighted 65 of 100 points, with a corroboration multiplier so a single unconfirmed provider flag scores meaningfully lower than several providers agreeing) and a correlation-graph-evidence component (35 of 100 points, drawn from `app/correlation/engine.py`'s graph and given the same corroboration treatment to resist a single source flooding it with distinct, uncorroborated edges). The AI is then handed this already-decided score as a given fact and asked only to narrate a verdict/rationale consistent with it — it can never invent or override the number, and the platform's own final-assessment validation re-checks the AI's output against the deterministic result before persisting it. Every persisted assessment records `SCORING_ENGINE_VERSION`, the exact version of this weighting scheme that produced it, so any historical score can always be traced back to the logic that generated it.

Celery: Scope and a Key Boundary

`celery_worker` and `celery_beat` both run the same task application, `app.workers.celery_app`. Exactly **one** scheduled task exists in the codebase: `run_osint_crawl`, triggered hourly by `celery_beat` (`backend/app/workers/celery_app.py`), which re-crawls OSINT (open-source intelligence) sources for IOCs that were looked up in the last 24 hours (`backend/app/workers/tasks.py`).

That is the entirety of what Celery does in this platform. **The main, user-facing IOC lookup pipeline — the synchronous, streamed lookup reachable at `POST /api/v1/lookup/stream` — does not go through Celery at all.** This is stated directly in a header

comment in `celery_app.py`. A lookup request (provider fan-out, per-provider AI summarization, correlation, final AI assessment, and persistence) is handled entirely within the FastAPI backend process for the duration of that one HTTP connection, with results streamed to the browser as Server-Sent Events (SSE) as each step completes. Celery is reached only by the separate, unrelated hourly re-crawl job — never as part of answering a live lookup request.

Deployment Paths

Two deployment paths are implemented and verified in the repository:

1. **Docker Compose** — `docker-compose.yml` (development) and `docker-compose.yml` + `docker-compose.prod.yml` (production). This is the path the Windows installer automates end-to-end (covered in the Windows Deployment / Installer section of this document).
2. **Kubernetes** — the `k8s/` directory contains a parallel, **Kustomize**-based Kubernetes deployment: StatefulSets for `postgres`, `neo4j`, and `opensearch`; Deployments for `backend`, `frontend`, and `celery`; and an Ingress resource. This is an alternative to the Docker Compose / Windows-installer path. Beyond its existence and the resource types listed above, further detail on this path (replica counts, resource limits, ingress rules, or how it is intended to be operated) is **Not confirmed** — it was not otherwise inspected as part of this documentation effort.

Component Diagram

The diagram below reflects verified relationships only. Solid arrows are real, active data paths confirmed in code; dashed arrows into `neo4j` and `opensearch` represent the config fields and running containers that exist but have no consuming application code.

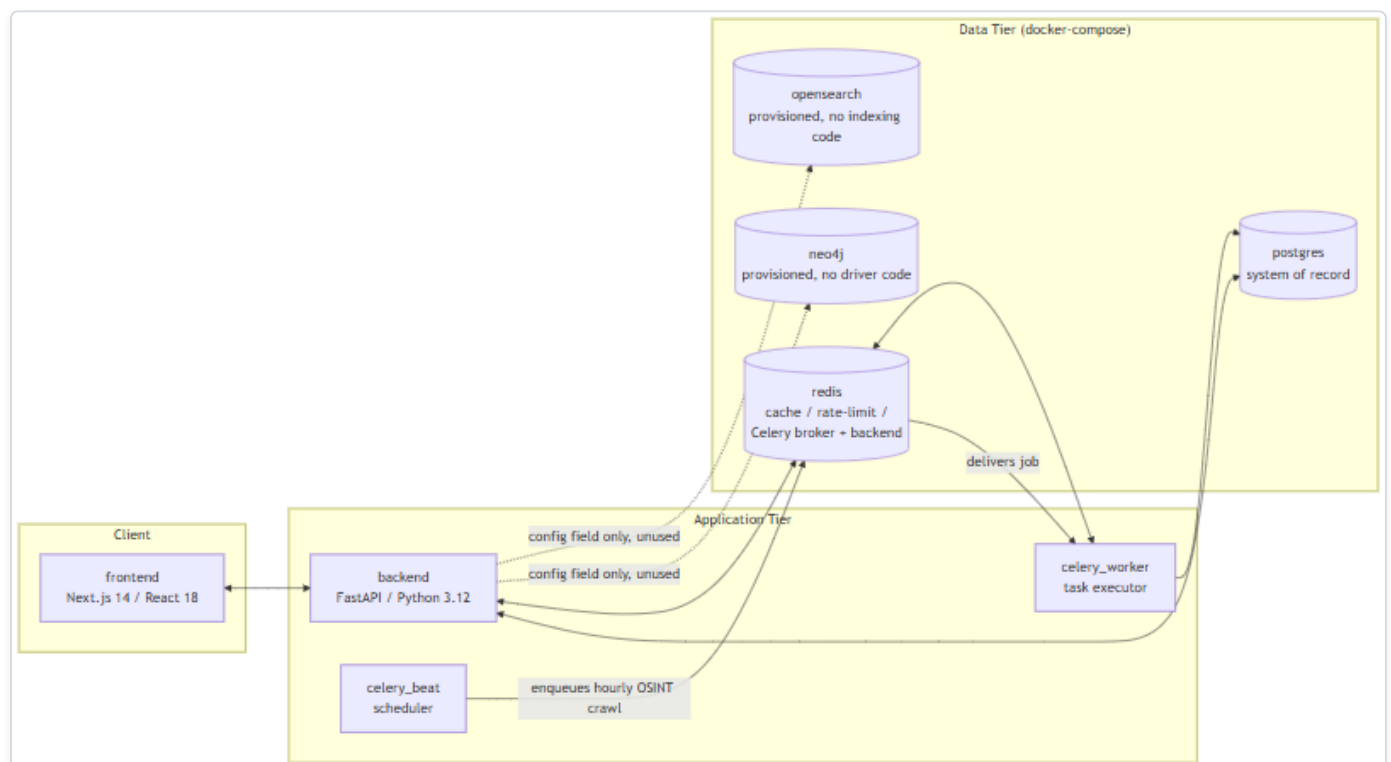


Figure 2 — Diagram: Component Diagram

Note what is *not* in this diagram: there is no edge from `backend` to `celery_worker` / `celery_beat` for the lookup pipeline, because none exists. The live IOC-lookup path is entirely `frontend` <-> `backend` <-> `postgres/redis`; the Celery containers are reached only

by the independent, hourly-scheduled OSINT re-crawl job described above.

Data Flow

This section traces exactly what happens, in code, when a user submits one IOC (Indicator of Compromise — a piece of evidence such as an IP address, domain, URL, file hash, CVE ID, or MITRE ATT&CK technique ID that a security analyst wants to investigate) to the platform, from the initial HTTP request through to the final "done" event. It then states plainly which of the platform's four datastores actually participate in that flow today, and which do not.

The entry point for a lookup is a single endpoint: `POST /api/v1/lookup/stream` (`backend/app/api/routes/lookup.py`). It is a Server-Sent Events (SSE) endpoint — the client opens one HTTP connection and receives a sequence of typed events as the lookup progresses, rather than blocking for a single response. The sequence below is verified directly against that endpoint's implementation.

The 9-Step Lookup Pipeline

1. **Rate limit check.** A Redis-backed fixed-window rate limiter (`RateLimiter`, `app/core/cache.py`) rejects the request if the calling user has exceeded the configured quota — 10 calls per 60 seconds per user by default (`app/core/config.py`), both values configurable.
2. **IOC type detection.** `detect_ioc_type()` (`app/ioc/detector.py`) classifies the raw input string using ordered regex and heuristic checks into one of 32 `IOCType` enum values (`app/ioc/types.py`) — for example, distinguishing an IPv4 address from a domain from a SHA256 hash.
3. **Lookup record created.** An `IOCLookup` row is created in Postgres immediately, with `status = RUNNING`. This is the row that will later be updated with the final verdict and risk scores.
4. **Provider fan-out.** `run_all_providers()` (`app/providers/orchestrator.py`) concurrently invokes every registered provider (of 16 total) whose `supports(ioc_type)` returns true for this IOC's type, using `asyncio.create_task` and `asyncio.as_completed` — providers run in parallel, not sequentially. A "provider" here is an external threat-intelligence, vulnerability, or OSINT source the platform queries (for example VirusTotal, AbuseIPDB, NVD). Each provider call passes through a Redis cache check first, then `BaseProvider.run()` (timeout and retry handled by `tenacity`, with configurable `provider_timeout_seconds` and `provider_max_retries`), producing a normalized `ProviderResult`.
5. **Per-provider persistence and summary.** As each provider result arrives (not waiting for all of them), it is persisted to Postgres as a `ProviderResultRecord`. If that provider's status is `ok`, `summarize_provider()` (`app/ai/service.py`) makes one AI call for that provider alone, grounded only in that provider's own JSON response, and the result is persisted as an `AISummaryRecord` tagged with that `provider_id`. Both the raw result and the summary are streamed to the client as SSE events (`provider_result`, `provider_summary`) and committed to Postgres after every single provider completes — a deliberate design so that a client disconnecting mid-lookup does not lose already-completed provider data (`lookup.py`, lines 122-136).
6. **Correlation.** Once every provider has finished, `correlate()` (`app/correlation/engine.py`) runs. This is a pure, I/O-free function: it extracts typed relationship edges from normalized provider fields (`resolved_ips`, `related_hashes`, `malware_families`, `mitre_techniques`, `cves`, and others), deduplicates and corroborates them — each additional provider that independently reports the same fact boosts its confidence score by a fixed increment (`_CORROBORATION_BONUS_PER_PROVIDER = 0.15`) — and returns a `CorrelationResult` (nodes, edges, deduplicated facts, provider agreement). The resulting edges are persisted as `CorrelationEdgeRecord` rows in Postgres and streamed as a `correlation` SSE event.
7. **Final AI assessment.** `generate_final_assessment()` (`app/ai/service.py`) makes exactly one consolidated AI call, grounded in all of the per-provider summaries plus the correlation output — never in the raw provider JSON directly. The result is validated against the `FinalAssessment` schema, cross-checked against the deterministic correlation data by `_ground_final_assessment()` (which silently strips any agreeing/disagreeing provider citation that isn't actually backed by real correlation data, and separately

flags — rather than removes — any cited MITRE technique that isn't backed by a real correlation edge, by setting that technique mapping's `grounded` field to `False`), and written onto the `IOCLookup` row as `final_verdict`, `risk_score`, `confidence_score`, and the full `final_assessment` JSON.

8. **Evidence ledger build.** `build_evidence()` (`app/evidence/builder.py`) deterministically converts the provider results, AI summaries, and correlation edges into `EvidenceItem` rows. This step is explicitly **not AI-generated** (per the module's own docstring) — it exists so that later AI-driven explanations elsewhere in the product can cite real, checkable evidence records instead of re-summarizing from memory.
9. **Final SSE events.** The stream emits a `final_assessment` event, then a `done` event, closing the connection.

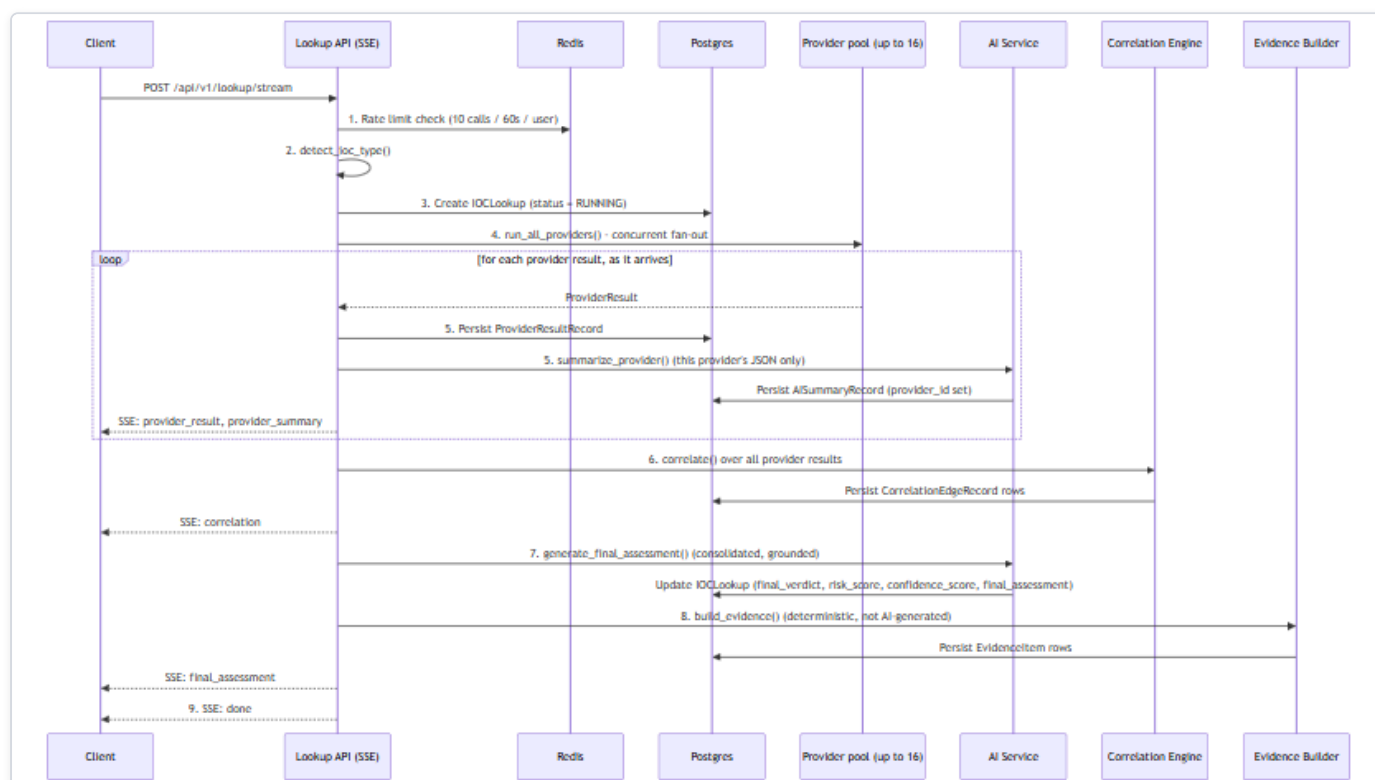


Figure 3 — Diagram: The 9-Step Lookup Pipeline

Where Data Actually Lives

The Docker Compose stack provisions four datastores — Postgres, Redis, Neo4j, and OpenSearch. Reading the pipeline above, it would be easy to assume all four are in play. They are not. This distinction matters enough that it is called out separately: **only Postgres and Redis have any application code that actually reads from or writes to them.** Neo4j and OpenSearch are running containers with nothing in the backend that talks to them.

Datastore	Provisioned in <code>docker-compose.yml</code> ?	Application code that uses it?	What it actually holds today
Postgres	Yes (<code>postgres:16-alpine</code>)	Yes	Everything: <code>IOCLookup</code> rows, <code>ProviderResultRecord</code> s, <code>AISummaryRecord</code> s (per-provider and final), <code>CorrelationEdgeRecord</code> s, <code>EvidenceItem</code> s, plus users, cases, and analyst baskets
Redis	Yes (<code>redis:7-alpine</code>)	Yes	Provider-result cache (keyed <code>provider_cache:{provider_id}:{ioc_type}:{sha256(value)}</code> , default TTL 3600s), the lookup-creation rate limiter, and the Celery broker/result-backend queues
Neo4j	Yes (<code>neo4j:5-community</code> + APOC plugin)	No	Nothing. Config fields only (<code>neo4j_uri</code> , <code>neo4j_user</code> , <code>neo4j_password</code>)
OpenSearch	Yes (<code>opensearchproject/opensearch:2.17.0</code>)	No	Nothing. A config field only (<code>opensearch_url</code>)

Postgres is the correlation graph today

Step 6 of the pipeline above talks about a "correlation graph" of nodes and edges between IOCs. It is tempting to assume that graph lives in Neo4j, since Neo4j is a graph database and is sitting right there in the stack. It does not. **The correlation graph is a set of rows in Postgres's `correlation_edges` table, queried relationally — full stop.** `CorrelationEdgeRecord` 's own docstring (`models/lookup.py`) and the correlation engine's module docstring (`correlation/engine.py`) both describe edges as being "mirrored into Neo4j," but that describes an intended future design, not current behavior. A full-repo search of `backend/app` for `neo4j` , `Neo4j` , or `GraphDatabase` (outside the Python virtual environment) turns up zero driver instantiations, zero Cypher queries, and zero read or write calls anywhere in the application. The only things Neo4j-related in the codebase are the three unused config fields and those two docstrings.

There is no search functionality

The same is true of OpenSearch, with even less ambiguity: there are no docstrings suggesting an intended integration, no indexing pipeline, and a full-repo search finds zero OpenSearch client, index, or search code anywhere in `backend/app` . The container runs; nothing in the product talks to it. Any full-text search a user might expect (e.g., searching historical lookups by free text) is not implemented against OpenSearch or anywhere else.

Practical takeaway

For a judge or engineer evaluating this build: treat Neo4j and OpenSearch as **provisioned infrastructure for a future phase**, not as working features. If asked "does this platform have a graph database backing its correlation view?" or "does it support full-text search?", the accurate answer is no — both containers exist and are healthy, but the entire correlation and evidence pipeline described in the nine steps above runs, end to end, on Postgres and Redis alone.

AI Architecture (Technical)

This section describes how HORIZON GRID's backend (`backend/app/ai/`) uses large language models to summarize and score an **IOC** (Indicator of Compromise — a hash, IP, domain, URL, CVE, etc. submitted for lookup) once raw provider data has been collected. It assumes the reader is technically literate but new to this codebase, so product-specific components — `ProviderResult` , `CorrelationResult` , `AISummaryRecord` , `IOCLookup` , `EvidenceItem` — are defined at first use.

All facts below are traceable to `backend/app/ai/service.py` , `schemas.py` , `analysis_service.py` , `hunting_service.py` , `analysis_schemas.py` , `ollama_client.py` , `anthropic_client.py` , `bedrock_client.py` , `gemini_client.py` , `groq_client.py` , `openai_client.py` , `kimi_client.py` , `deepseek_client.py` , `xai_client.py` , `mistral_client.py` , and `openrouter_client.py` .

1. Supported AI Backends and Selection

The active backend is a single configuration value, `ai_backend` (`app/core/config.py` line 69), defaulting to `"ollama"` . Eleven backends are implemented:

Backend	Config value	Mode	Auth / connection detail
Ollama	<code>ollama</code>	Local, default	No key; talks to <code>http://host.docker.internal:11434</code> , model <code>llama3.2:3b</code>
Anthropic	<code>anthropic</code>	Direct API	Direct Messages API, tool-forced JSON output
AWS Bedrock	<code>bedrock</code>	Cloud	Bedrock Converse API for Claude; supports bearer-token or IAM access-key/secret auth
Google Gemini	<code>gemini</code>	Cloud	Gemini <code>generateContent</code> REST endpoint; JSON-schema response mode
Groq	<code>groq</code>	Cloud, fast inference	OpenAI-compatible chat-completions API at <code>api.groq.com</code> ; Bearer-token auth; default model <code>llama-3.3-70b-versatile</code>
OpenAI	<code>openai</code>	Cloud	OpenAI's own chat-completions API at <code>api.openai.com</code> ; Bearer-token auth; forced tool-calling for structured output
Kimi (Moonshot AI)	<code>kimi</code>	Cloud	OpenAI-compatible chat-completions API at <code>api.moonshot.ai</code> ; Bearer-token auth
DeepSeek	<code>deepseek</code>	Cloud	OpenAI-compatible chat-completions API at <code>api.deepseek.com</code> ; Bearer-token auth
xAI	<code>xai</code>	Cloud	OpenAI-compatible chat-completions API at <code>api.x.ai</code> ; Bearer-token auth
Mistral	<code>mistral</code>	Cloud	Chat-completions API at <code>api.mistral.ai</code> ; Bearer-token auth; forced tool-calling for structured output
OpenRouter	<code>openrouter</code>	Cloud, meta-router	OpenAI-compatible chat-completions API at <code>openrouter.ai</code> , routing to many underlying model vendors; Bearer-token auth

All eleven client classes expose the **same** async method signature — `call_claude_json(system_prompt, user_prompt, json_schema, tool_name, max_tokens)` — so `app/ai/service.py` 's internal `_get_ai_client()` never has to branch on which backend is active (service.py lines 42-78). This is a straightforward adapter pattern: swapping backends is a config change, not a code change. `groq_client.py` and the five clients modeled on it (`openai_client.py` , `kimi_client.py` , `deepseek_client.py` , `xai_client.py` , `mistral_client.py` , `openrouter_client.py`) fit this pattern by targeting their vendor's own OpenAI-compatible chat-completions endpoint and using the same forced-single-tool-call technique the other backends use to coerce a structured JSON response, rather than a bespoke prompt format.

Model-list handling now splits into two groups rather than "Groq vs. everyone else": Anthropic, Gemini, and Bedrock still use a hardcoded/static model list in the codebase. Every other backend discovers its model list **live** instead — Groq, OpenAI, Kimi, DeepSeek, xAI, Mistral, and OpenRouter each query their own vendor `GET /models` endpoint (exposed to the setup wizard and API consumers via the shared `POST /api/v1/ai/{backend}/models` route), and Ollama lists whatever is actually pulled locally via `GET {base_url}/api/tags`. For the seven vendor-hosted live-discovery backends, this means the model dropdown always reflects the account's real, currently-available models rather than a list that can silently go stale as a vendor deprecates or introduces models.

Selection is **fail-fast**: if the currently selected backend's `is_configured` check returns `False` (e.g. `anthropic` selected with no API key present), the service raises a `RuntimeError` immediately rather than letting the request proceed into a confusing low-level HTTP failure.

2. Runtime AI Backend Switching (No Restart)

Until this session, `ai_backend` and every backend's credentials were read once, at process start, from `.env` via `get_settings()` (`app/core/config.py`) — a `@lru_cache`-wrapped singleton that lives for the lifetime of the container process. Switching from Ollama to Anthropic, or adding a Groq key, meant editing `.env` and restarting the Docker containers; there was no way to change the active AI backend or its credentials while the platform was running.

`_get_ai_client()` in `service.py` now resolves the active backend on **every call**, rather than once at startup, by reading a database-backed runtime-config service (`backend/app/core/runtime_config.py`) instead of the frozen Settings object. Resolution order is: (a) an explicit `backend_override`, used only by the AI-comparison feature described below; (b) the currently **active** runtime-configured backend — a fresh row read from the `provider_runtime_configs` table (kind `"ai"`, `is_active = true`) on that call; (c) the legacy Settings-based value, kept only as a fallback for installs where no runtime config has been seeded yet. Each of the eleven AI client classes (`ollama_client.py`, `anthropic_client.py`, `bedrock_client.py`, `gemini_client.py`, `groq_client.py`, `openai_client.py`, `kimi_client.py`, `deepseek_client.py`, `xai_client.py`, `mistral_client.py`, `openrouter_client.py`) gained optional constructor parameters so `_get_ai_client()` can build a fresh, correctly-credentialed instance from the decrypted runtime credentials on each call instead of reusing a stale cached singleton built from `.env` at process start.

The practical effect: `POST /api/v1/runtime/ai-active` switches the active backend, and the very next AI call — the next per-provider summary or final assessment, for any in-flight or new lookup — uses the newly active backend and its credentials. No container restart, no `.env` edit, no redeploy.

This same on-demand resolution also underlies the AI-comparison feature: `POST /api/v1/lookup/{id}/reanalyze` re-runs only the `generate_final_assessment()` step against an already-completed lookup's existing evidence, passing a `backend_override` for whichever backend the analyst wants to compare against — no provider is re-queried. Every assessment ever produced for a lookup — the original plus every later comparison run — is written to a new `final_assessment_records` table (`app/models/lookup.py`), tagged with the `ai_backend` / `ai_model` that produced it and an `is_primary` flag distinguishing the original from comparisons; a reanalysis never overwrites or deletes an earlier result. `GET /api/v1/lookup/{id}/assessments` returns the full set for side-by-side review.

One further prompt-level change accompanies this: `generate_final_assessment()` now also receives an explicit `unavailable_providers` list — providers that were applicable to the IOC but failed, timed out, were rate-limited, or were not configured/disabled for this run — rendered in the prompt as a distinct "Providers unavailable this run" section, together with a system-prompt rule never to read an unavailable provider's absence as "found nothing" or "reported clean." This closes a gap that predates runtime switching but became more visible once providers could be disabled live: without it, the AI had no way to distinguish missing coverage from a genuinely clean result.

3. Two Distinct AI Call Types

The pipeline makes two structurally different kinds of AI calls, confirmed as genuinely separate in the code (not a naming convention over one code path):

	<code>summarize_provider()</code>	<code>generate_final_assessment()</code>
Cardinality	One call per provider that returned OK data	One call per lookup, after all providers finish
Input	Only that single provider's own JSON	All per-provider <code>ProviderSummary</code> objects + the correlation engine's output — never raw provider JSON
Stored as	<code>AISummaryRecord</code> with <code>provider_id</code> set	<code>IOCLookup.final_assessment</code> and an <code>AISummaryRecord</code> with <code>provider_id = NULL</code>

The `AISummaryRecord.provider_id` column is the actual database-level marker distinguishing the two: a model-level comment (`models/lookup.py` line 93) explicitly documents that a `NULL provider_id` row is "the final consolidated assessment, not a per-provider one."

Why they are kept separate, per the code's own design comments (`service.py` lines 8-11): the final assessment is deliberately grounded in the already-summarized per-provider outputs and the deterministic correlation result rather than being handed every provider's raw JSON again. This bounds the size of the final prompt (which would otherwise grow with the number of registered providers) and forces the "master" call to reason over an already-distilled, per-provider-attributed layer instead of re-deriving everything from scratch — which is also what makes the grounding/cross-check step in \$5 possible.

4. No-Evidence Short-Circuit Guard

Inside `generate_final_assessment()` (`service.py` lines 242-284), there is a hard guard: if `provider_summaries` is empty **and** `correlation.edges` is empty, the function returns a hardcoded, deterministic `FinalAssessment` — `final_verdict=Verdict.UNKNOWN`, all risk fields set to `0` — **without invoking the AI backend at all**.

The code comment documents the concrete, reproduced failure that motivated this guard: querying the real MD5 hash of the EICAR antivirus test file (`44d88612fea8a8f36de82e1278abb02f`) with **zero configured providers** — i.e. with no actual evidence of any kind — still caused the small local model (`llama3.2:3b`) to return `final_verdict="highly_malicious"` and `malicious_probability=92`, fabricating an "association with ransomware and trojans" purely from its pretrained knowledge. This directly violated the model's own system-prompt instruction to never fabricate findings. Rather than trying to prompt-engineer around this failure mode, the team chose to bypass the model entirely whenever there is structurally no evidence to reason over — a deterministic guard is unconditionally reliable where a prompt instruction was not.

5. Grounding / Cross-Check Mechanism

Even when the AI backend *is* called, its structured output is not trusted at face value — it is cross-checked against the deterministic correlation data. Two related mechanisms implement this:

- `_ground_final_assessment()` (`service.py` lines 137-190), run on every `generate_final_assessment()` result:
- Filters the model's `agreeing_providers` / `disagreeing_providers` lists down to only the `provider_id` s that actually appear in the correlation edges, in `provider_agreement`, or in `known_provider_ids` (the set of providers that genuinely returned a per-provider summary, passed in by the caller). Anything else the model names is **silently stripped**.
 - Sets `mapping.grounded = False` on any MITRE ATT&CK technique the model cited that is not backed by a real `uses_technique` correlation edge — so a technique mention that has no supporting evidence is flagged rather than presented as fact.

A parallel mechanism in `analysis_service.py`, applied to the analyst-facing explanation endpoints (WHY malicious, Challenge/red-team, Copilot Q&A, and others):

- `_strip_invalid_evidence_ids()` (lines 55-71) recursively removes any `evidence_ids` citation that doesn't match a real `EvidenceItem.id` belonging to that specific lookup.
- `_backfill_evidence_ids_from_prose()` (lines 74-112) does the reverse repair: it recovers citations the model correctly referenced in free-text prose but failed to place into the structured `evidence_ids` field — but it only ever adds an ID that has already been confirmed real; it never invents one.

Together, these mean every AI-authored claim that reaches storage or the UI has either been matched against a real `EvidenceItem` /correlation edge or explicitly marked/stripped as unsupported — the deterministic correlation and evidence layers act as a check on the generative layer, not the other way around.

6. Verdict and Confidence Data Model

The `Verdict` enum (`app/models/lookup.py` lines 22-34) has exactly **12 values**: `highly_malicious`, `malicious`, `suspicious`, `unknown`, `likely_benign`, `benign`, `scanner`, `tor_exit_node`, `vpn`, `cdn`, `cloud_infrastructure`, `dormant_infrastructure`. Note that several of these are not "maliciousness" gradations at all but infrastructure classifications (`tor_exit_node`, `vpn`, `cdn`, `cloud_infrastructure`, `dormant_infrastructure`) — the model is expected to distinguish "this is malicious" from "this is a category of infrastructure that merits a different interpretation."

`RiskAssessment` (`app/ai/schemas.py` lines 101-131) carries `overall_risk_score`, `confidence_score`, and `malicious_probability`, all explicitly on a **0-100 scale** (not 0-1). A `field_validator` automatically rescales any value it receives between 0 and 1 by multiplying by 100 — a defensive fix added because `llama3.2:3b` was observed defaulting to a 0-1 convention despite the 0-100 instruction.

`FinalAssessment` additionally carries a `model_validator` named `_verdict_must_agree_with_risk` (`schemas.py` lines 187-208) that rejects egregiously self-contradictory combinations — for example, a `final_verdict="malicious"` paired with `malicious_probability < 30` is rejected rather than silently persisted. This is a second, independent layer of internal-consistency checking, separate from the evidence-grounding checks in §5.

`FinalAssessment` also carries two traceability fields, `ai_backend` and `ai_model` (`schemas.py`), populated by `service.py` at generation time and surfaced in the UI as a badge on the final-assessment panel. With eleven interchangeable backends now selectable, and the config free to change between one lookup and the next, these fields exist so that an analyst or judge reviewing a conclusion always knows exactly which provider and model produced it — never a silent or ambiguous backend switch. When the no-evidence guard in §4 fires, or a generation error occurs (as happens on a rate-limited call), both fields are left `null` rather than populated with a misleading value, consistent with this layer's broader rule of never presenting a fabricated or guessed fact as if it were real.

7. AI Data Flow

The diagram below shows data flowing *into* the AI layer for a single lookup: per-provider JSON feeds per-provider summaries; those summaries plus the correlation engine's output feed the one consolidated final assessment; the final assessment is grounded/cross-checked before being persisted.

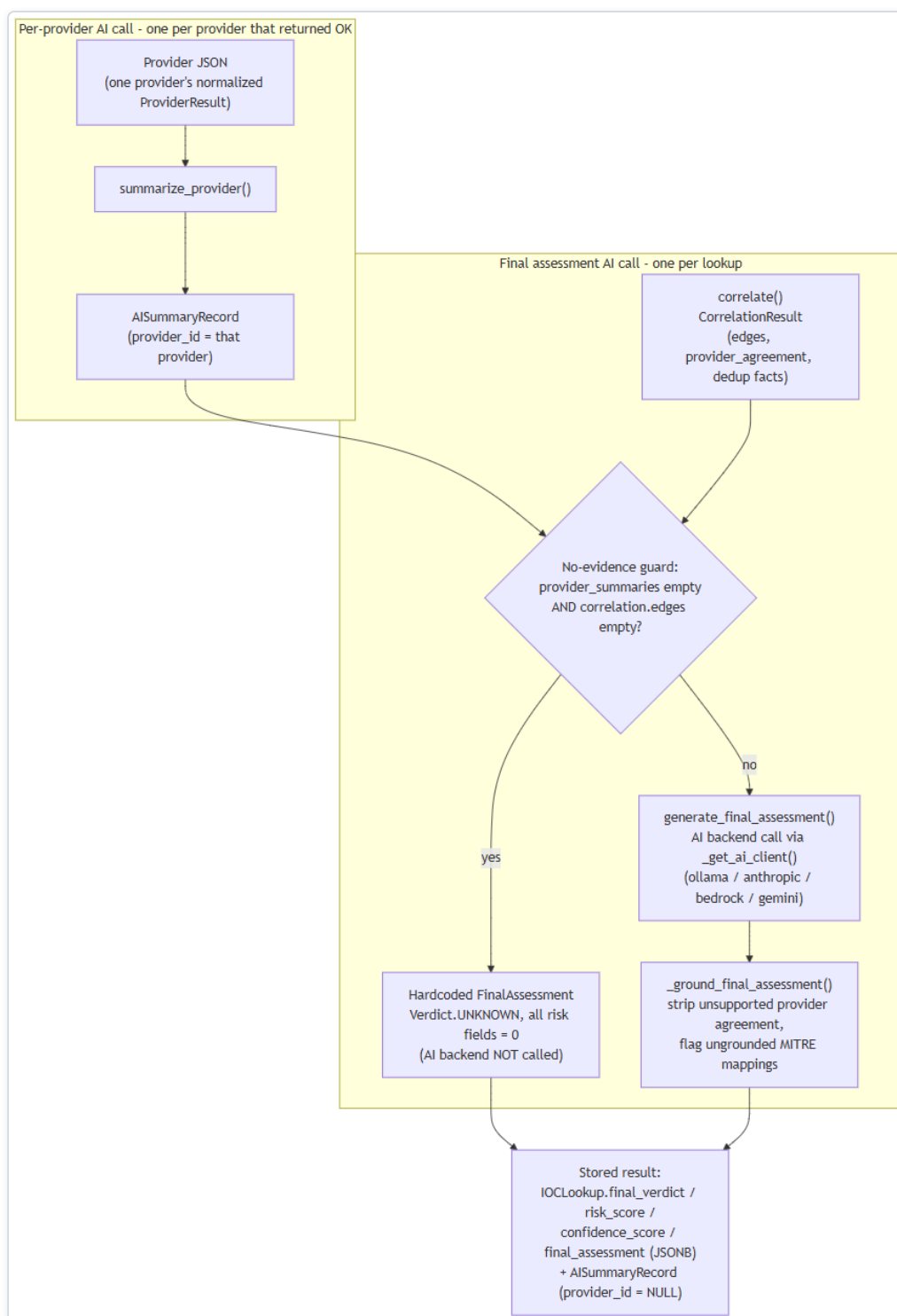


Figure 4 — Diagram: 7. AI Data Flow

8. Beyond the Core Pipeline

`analysis_service.py` implements eight further grounded, evidence-cited AI features on top of the same call/grounding pattern: WHY malicious, What Is This, Provider Disagreement, False-Positive Assessment, Challenge/red-team, Smart Next Actions, Intelligence Gaps, and Score Explanation, plus Copilot Q&A and IOC comparison. `hunting_service.py` implements hunting-query and detection-

rule generation. Both are restricted to citing only indicators/evidence that genuinely exist in the correlation graph or evidence ledger for that lookup — `hunting_service.py` (lines 74-79) explicitly strips any invented expansion target the model produces, following the same "never trust generated output at face value" principle applied throughout this layer.

Provider Architecture

A **provider** is a self-contained connector class in the platform's backend that queries one external data source — a threat-intelligence feed, a vulnerability database, a DNS blocklist, a certificate-transparency log, a sandbox scanner, or an OSINT (open-source intelligence) crawler — for information about a single IOC (indicator of compromise) submitted to a lookup. All 18 providers shipped with the platform are registered in `backend/app/providers/registry.py` (`_ALL_PROVIDERS`) and share one common interface, described below.

Common Provider Interface

Every provider implements the same three-part contract, so the orchestrator (the component that fans a lookup out to providers — see below) never needs provider-specific logic:

- **`supports(ioc_type)`** — a boolean check for whether this provider is eligible to run against the specific kind of indicator submitted. `ioc_type` is one of 31 values in the platform's `IOCType` enum (`app/ioc/types.py`), assigned to the raw input by the IOC-type detector before any provider runs. A provider is only invoked if `supports()` returns true for the lookup's detected type (e.g. `hybrid_analysis` only supports `sha256`; `whois_rdap` supports `domain`, `ipv4`, `ipv6`, and `asn`).
- **`configured`** — a boolean property reporting whether the provider currently has what it needs to run (typically a credential). Some providers are unconditionally configured because they require no credential at all — for example `nvd` has `requires_key = False` and `configured = True` unconditionally. Others require specific fields to be present — for example `censys` is only `configured` when **both** a Personal Access Token and an Organization ID are set; either alone leaves it not configured.
- **`run()`** (via the shared `BaseProvider.run()`) — executes the actual outbound call (HTTP request, DNS query, etc.) and returns a normalized `ProviderResult`. This call is wrapped with a timeout and retry policy applied uniformly across all providers: `provider_timeout_seconds` (default 20 seconds) and `provider_max_retries` (default 2), with exponential backoff implemented via the `tenacity` library (`wait_exponential(multiplier=0.5, max=4)`).

Before `run()` is invoked, each provider call first checks the Redis cache layer (described below); a cache hit skips the outbound call entirely.

Concurrent Fan-Out Execution Model

For a given lookup, `run_all_providers()` (`backend/app/providers/orchestrator.py`) does **not** call providers one after another. It fans out **concurrently** to every registered provider whose `supports(ioc_type)` is true, using `asyncio.create_task` to launch all eligible provider calls at once and `asyncio.as_completed` to consume each result as soon as it finishes, in whatever order calls actually complete. Each provider call independently goes through: Redis cache check → `BaseProvider.run()` (timeout/retry as above) → normalized `ProviderResult`. As each result arrives, it is persisted to Postgres and streamed to the client immediately (rather than the pipeline waiting for every provider to finish before persisting or streaming anything).

The platform's integration test suite exercises this concurrency directly against fake provider stubs (never the real network-hitting connectors), verifying parallel — not sequential — fan-out, isolation of one provider's failure from the others, cache hit/miss behavior, and correct short-circuiting of unsupported or not-configured providers.

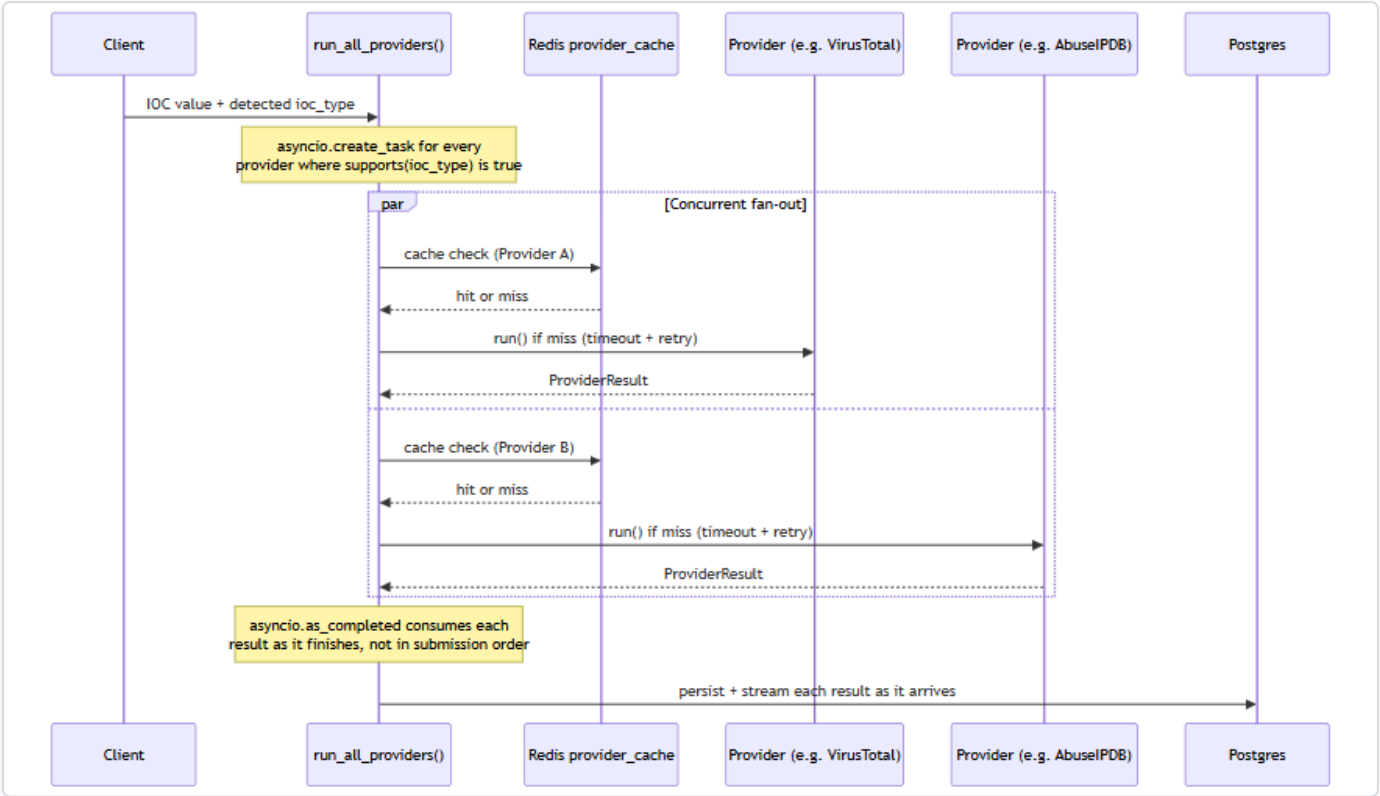


Figure 5 — Diagram: Concurrent Fan-Out Execution Model

"Provider A" and "Provider B" above are illustrative stand-ins for any two of the up to 18 registered providers eligible for a given IOC type — the fan-out is not limited to two.

Redis Caching Layer

Provider results are cached in Redis to avoid repeating identical outbound lookups. The cache key format is `provider_cache:{provider_id}:{ioc_type}:{sha256(value)}` (`app/core/cache.py`), and the time-to-live is `provider_cache_ttl_seconds`, which defaults to 3600 seconds (1 hour). This provider-result cache lives on Redis logical DB index `/0`, kept separate from the Celery broker (`/1`) and Celery result backend (`/2`) that run on the same Redis instance.

Two providers additionally maintain their own cache on top of this (visible in the Notes column of the table below): `cisa_kev` keeps an in-memory cached copy of its catalog with a 1-hour TTL, and `mitre_attack` caches its STIX bundle for 1 hour. These are provider-internal caches, distinct from the shared Redis provider-result cache described above.

The 18 Registered Providers

Provider	Category	Supported IOC types	Key requirement	Notes
<code>virustotal</code>	threat_intel	ipv4, ipv6, domain, url, hashes	API key	v3 free tier
<code>abuseipdb</code>	threat_intel	ipv4, ipv6	API key	v2
<code>otx</code>	threat_intel	ipv4, ipv6, domain, hostname, url, hashes	API key	AlienVault OTX
<code>urlhaus</code>	threat_intel	url, domain, ipv4	abuse.ch Auth-Key	shares key w/ threatfox/malwarebazaar
<code>threatfox</code>	threat_intel	ipv4, ipv6, domain, url, md5, sha256	abuse.ch Auth-Key	shares key
<code>malwarebazaar</code>	threat_intel	hash types (md5/sha1/sha256/sha512)	abuse.ch Auth-Key	shares key
<code>crtsh</code>	certificate_intel	domain, tls_certificate	none	Certificate Transparency
<code>nvd</code>	vulnerability	cve	none (optional key raises rate limit)	NIST NVD v2.0
<code>cisa_key</code>	vulnerability	cve	none	in-memory cached catalog, 1hr TTL
<code>mitre_attack</code>	threat_intel	mitre_technique	none	STIX bundle, cached 1hr
<code>whois_rdap</code>	whois	domain, ipv4, ipv6, asn	none	python-whois (domains) + RDAP (IP/ASN)
<code>urlscan</code>	sandbox	url, domain	API key	urlscan.io; submit-then-poll scan model (<code>POST /scan/</code> , then poll <code>GET /result/{uuid}/</code> , which returns HTTP 404 while still processing); the poll loop is wall-clock timeboxed and reports a timeout rather than fabricating a result from an incomplete scan
<code>google_safe_browsing</code>	threat_intel	url, domain	API key	Google Safe Browsing Lookup API v4 (<code>threatMatches:find</code>); a confirmed HTTP 200 with an empty/absent <code>matches</code> field is the <i>only</i> input that may ever produce a "clean" verdict
<code>hybrid_analysis</code>	sandbox	sha256 only	API key	v2 overview/summary endpoint; MD5/SHA1/URL explicitly unsupported per code comment (deprecated endpoints)
<code>spamhaus</code>	threat_intel	ipv4, domain	none	DNSBL lookup, no HTTP call (raw DNS)
<code>phishtank</code>	threat_intel	url	none (optional key raises rate limit)	
<code>censys</code>	passive_dns	ipv4, ipv6	Personal Access Token and Organization ID both required	Platform API v3
<code>internet_intelligence</code>	osint	domain, ipv4, malware_family, threat_actor, campaign, cve,	none	OSINT crawler wrapping GitHub/Reddit/RSS/pastebin sources

Provider	Category	Supported IOC types	Key requirement	Notes
		file_name		(app/crawler/)

A deliberately tested safety guarantee, specific to these two newest providers: both `urlscan` and `google_safe_browsing` are built so that a genuine failure — a rejected/invalid API key, an unreachable API, or a rate limit — is always reported as an error or unknown status, never as a false "clean"/"safe" result. `google_safe_browsing.py`'s own module docstring states the invariant directly: an empty or missing `matches` field on a genuine HTTP 200 response is the *only* input that may ever produce a "clean" verdict; every other outcome — a non-200 status, a network error or timeout, or a response body that doesn't parse the way the API contract promises — routes through the same error-handling path, which always sets a `{"verdict": "unknown", ...}` payload rather than anything that could be mistaken downstream for a real clean scan. This was confirmed with a real, live chaos test that supplied deliberately invalid credentials to both providers' real external APIs: the invalid key was rejected with `status=error`, Safe Browsing's own verdict field read `"unknown"` (never `"clean"`), and the investigation's overall verdict was reported as `UNKNOWN` rather than benign or clean.

Wizard Coverage and the "6 Providers with No UI" Discrepancy

The Windows setup **wizard** (`windows/wizard/Setup-Wizard.ps1`, `$ProviderDefs`) presents a Provider Configuration screen listing exactly 8 entries: `virustotal`, `abuseipdb`, `otx`, `abusech` (a single combined entry covering the shared abuse.ch key used by `urlhaus`, `threatfox`, and `malwarebazaar`), `nvd`, `hybrid_analysis`, `censys`, and `phishtank`. Counting the three abuse.ch-backed connectors individually, those 8 wizard entries map to 10 of the platform's 16 backend providers. A sample of the wizard's entries was independently checked against the corresponding provider's code and matched exactly — for example, the wizard's Censys note ("Requires both a Personal Access Token and an Organization ID") matches `censys.py`'s configured check precisely, and the wizard's abuse.ch note ("one free Auth-Key covers all three connectors") matches all three connectors reading the same `abusech_auth_key` setting.

The remaining 6 backend providers — `crtsh`, `cisa_key`, `mitre_attack`, `whois_rdp`, `spamhaus`, and `internet_intelligence` — have real, working backend implementations but **do not appear anywhere in the wizard's UI**. This is a documented discrepancy between the wizard and the backend provider registry, but it is flagged as **consistent behavior, not a bug**: all six of these providers have `requires_key = False` — there is no credential for a user to enter in the first place. Because the wizard's Provider Configuration screen exists specifically to collect and test credentials, a provider with nothing to configure has nothing to show there. This is corroborated on the backend side: `backend/app/providers/connection_test.py`'s handler dictionary (which powers the wizard's live "Test" button, `POST /api/v1/providers/{id}/test`) likewise defines no test handler for these six providers, falling back to a message that explicitly identifies them as requiring no credential.

In short: 8 wizard entries cover 10 providers that have a credential-related field to configure — 8 of those (`virustotal`, `abuseipdb`, `otx`, and the three abuse.ch-backed connectors, plus `hybrid_analysis`, `censys`) require a key to function at all, while 2 (`nvd`, `phishtank`) only accept an optional key to raise their rate limit rather than strictly needing one. The other 6 providers run unconditionally with no credential of any kind and correctly have no wizard entry and no connection-test handler.

Runtime IOC Provider Configuration (No Restart)

Everything described above — the `configured` property, the credential fields, and the enabled/disabled state of each of the platform's registered providers — was, prior to this session, entirely `.env`-driven and frozen for the lifetime of the running process: `app/core/config.py`'s `get_settings()` is a process-lifetime `@lru_cache` singleton, so changing a provider's key or toggling it off meant editing `.env` and restarting the Docker containers before the change had any effect.

That has been replaced with a database-backed runtime layer. A new `provider_runtime_configs` table (`backend/app/models/runtime_config.py`, kind `"ioc"`) holds one row per provider — `enabled`, `encrypted_credentials` (Fernet-

encrypted, never plaintext), `extra_config`, and the timestamp/outcome of its last connection test. Two endpoints mutate this table: `POST /api/v1/runtime/ioc-providers/{id}` configures a provider's credentials, and `POST /api/v1/runtime/ioc-providers/{id}/enabled` flips it on or off. Neither requires touching `.env` or restarting anything — the very next investigation submitted after the call picks up the change.

Because each provider connector in `registry.py` is instantiated once and reused as a long-lived singleton for the life of the process, mutating a shared instance attribute on that singleton immediately before use would be unsafe: two investigations running concurrently — one started right after an administrator changes a provider's credentials or enabled state, while another is still mid-flight — could race and see each other's configuration. Instead, `run_all_providers()` (`backend/app/providers/orchestrator.py`) takes a single, immutable snapshot of runtime configuration — `{provider_id: {enabled, configured, credentials}}` — once per investigation, at the start of the fan-out, and holds it in a Python `ContextVar` rather than on the provider object itself. Because `asyncio.create_task()` copies the current context into every task it spawns, each concurrent per-provider task launched for that investigation reads from that same frozen snapshot, regardless of what an administrator does to the underlying table while those tasks are still running. Two investigations in flight at once, each with a different configuration for the same provider, therefore never see each other's values — verified with a dedicated concurrency test simulating exactly that scenario.

The provider-availability model also gained a new state to describe this. Previously a provider was either eligible (`supports()` true) and `configured`, or not; `ProviderStatus` now also has a `DISABLED` value, distinct from `NOT_CONFIGURED` — a provider an administrator has deliberately turned off via the enabled/disabled endpoint reports `DISABLED`, while a provider with no credential entered at all still reports `NOT_CONFIGURED`. Both are, in turn, distinguished from "not applicable to this IOC type," so the final-assessment AI prompt never conflates "unavailable" with "checked and found nothing."

Finally, `POST /api/v1/lookup/stream` gained an optional `provider_ids` field: passing a list of provider IDs restricts that one investigation to exactly those providers, regardless of how many are otherwise enabled and configured platform-wide, without affecting any other investigation.

The Same Live-Test Pattern, Extended to AI Backends

The `POST /api/v1/providers/{id}/test` mechanism described above — the one behind each provider's live wizard "Test" button — has a direct counterpart for the AI layer. Until this session, none of the platform's AI backends (Ollama, Anthropic, Bedrock, Gemini, or Groq — see the *AI Architecture* chapter for the backends themselves) had any live connection test at all; a new `POST /api/v1/ai/test` endpoint (`backend/app/api/routes/ai_config.py`, `backend/app/ai/connection_test.py`) now gives every one of them a real "Test Connection" button in the wizard's AI Configuration screen, mirroring the provider-side pattern rather than introducing a new one.

The same design rule that governs the provider-side tester carries over deliberately: the endpoint always tests the *candidate* credentials currently entered in the form, never the application's stored/active configuration. Because the test path never reads from or writes to the saved config, trying out a not-yet-saved key can never race with — or accidentally disturb — whatever backend and key the running platform is actually using for real investigations.

It is also, deliberately, not a generic pass/fail. The endpoint sends a minimal real request to the backend under test and reports one of several specific, genuine outcomes: authenticated success (with real round-trip latency and the exact model name that replied), invalid/rejected key, rate-limited, model-not-found, network error, or timeout — the same level of specificity the provider-side test handlers already establish, applied to the AI layer for the first time.

Database Architecture

HORIZON GRID's `docker-compose.yml` provisions four datastore containers: `postgres` (postgres:16-alpine), `redis` (redis:7-alpine), `neo4j` (neo4j:5-community with the APOC plugin), and `opensearch` (opensearchproject/opensearch:2.17.0). This section describes what each one actually does in the running system, based on direct inspection of the backend code rather than the docker-compose comments or model docstrings that describe the *intended* design. Two of the four -- Neo4j and OpenSearch -- are running containers with no backend code that reads from or writes to them; that gap is documented plainly below rather than glossed over.

An **IOC** (Indicator of Compromise) is the value a user submits for lookup -- an IP address, domain, file hash, URL, CVE ID, etc. A **provider** is a connector to an external threat-intelligence source (VirusTotal, AbuseIPDB, and so on) that the platform queries for a given IOC.

PostgreSQL: the system of record

PostgreSQL is the only datastore the application actually writes to during a lookup. Every table below is defined under `backend/app/models/`:

Table	Holds
<code>users</code>	Account records: email, hashed password, full name, role (<code>admin</code> / <code>analyst</code> / <code>viewer</code>), active flag.
<code>ioc_lookups</code>	One row per lookup: the submitted IOC value and type, a status enum (<code>pending</code> / <code>running</code> / <code>completed</code> / <code>failed</code>), and the final AI output -- <code>final_verdict</code> , <code>risk_score</code> , <code>confidence_score</code> , and the full <code>final_assessment</code> as JSONB.
<code>provider_results</code>	One row per provider call per lookup: that provider's raw response as JSONB, plus its status and latency.
<code>ai_summaries</code>	AI-generated summaries as JSONB. A nullable <code>provider_id</code> distinguishes a per-provider summary from the single final consolidated assessment (null <code>provider_id</code> marks the latter).
<code>correlation_edges</code>	The relationship graph produced by the correlation engine: source/target type and value, relationship type, a confidence score, and provenance (which provider(s) contributed the edge).
<code>evidence_items</code>	The deterministic, non-AI-generated evidence ledger: an evidence-type enum (9 values), a source label, a claim, an interpretation, a confidence score (0-100), and the underlying raw data as JSONB.
<code>basket_items</code>	Per-analyst scratch lists of IOCs, unique per owner and IOC value.
<code>cases</code> , <code>case_iocs</code> , <code>case_notes</code> , <code>case_reports</code>	A lightweight case-management workflow: cases with status/severity enums, the IOCs attached to a case, and analyst notes. <code>case_reports</code> exists as a table, but report generation itself is not implemented -- the field is present and permanently empty rather than populated by any code path.

Two things worth calling out because they matter for how the rest of this appendix should be read:

- **The correlation graph lives in Postgres.** `correlation_edges` is a plain relational table, populated by the correlation engine after every provider has returned, and streamed to the UI as part of the lookup. There is no separate graph database behind it today -- see the Neo4j section below.
- **The evidence ledger is deliberately not AI-generated.** `evidence_items` rows are built by a deterministic conversion step so that later AI-written explanations have real, checkable records to cite against, rather than citing their own prior output.

Schema changes are managed with **Alembic** (`backend/alembic/`), SQLAlchemy's migration tool. The application itself uses SQLAlchemy 2.0 in async mode over `asyncpg`.

Neo4j: provisioned, not integrated

Neo4j (`neo4j:5-community` , with the APOC plugin) runs as a container in `docker-compose.yml` , and the backend config (`app/core/config.py`) has `neo4j_uri` , `neo4j_user` , and `neo4j_password` settings for it. Some code comments describe an intended design where correlation edges get "mirrored into Neo4j" -- this appears in the `CorrelationEdgeRecord` model docstring and in the correlation engine's module docstring.

That design was never built. A full search of the backend application code for `neo4j` , `Neo4j` , or `GraphDatabase` (excluding the Python virtual environment) turns up zero driver instantiation, zero Cypher queries, and zero read or write calls anywhere in the application. Neo4j is not used for anything today. **The correlation graph described above is, in its entirety, the `correlation_edges` table in PostgreSQL.** Anyone evaluating this platform should treat Neo4j as reserved-but-idle infrastructure, not as an active graph store.

OpenSearch: provisioned, not integrated

The same pattern holds for OpenSearch (`opensearchproject/opensearch:2.17.0` , running with `DISABLE_SECURITY_PLUGIN: "true"`). The backend config has an `opensearch_ur1` setting, and the container runs as part of the stack, but a full search of the backend application code found zero OpenSearch client code, zero index definitions, and zero search calls. No indexing pipeline exists, and no user-facing search feature is backed by it. OpenSearch should be described as provisioned infrastructure only -- not as an active search index.

Neo4j and OpenSearch represent the most significant gap between the design implied by the docker-compose file and docstrings, and what the running system actually does. Both are real containers consuming real resources; neither currently does any work for the platform.

Redis: three real uses

Unlike Neo4j and OpenSearch, Redis (`redis:7-alpine`) is genuinely load-bearing, in exactly three ways -- all confirmed against `app/core/cache.py` , `docker-compose.yml` , and `app/core/config.py` :

1. **Provider-result cache.** Each provider call is cached under a key of the form `provider_cache:{provider_id}:{ioc_type}:{sha256(value)}` , with a TTL controlled by `provider_cache_ttl_seconds` (default 3600 seconds / 1 hour). A repeat lookup for the same IOC within the TTL window skips the live provider call.
2. **Rate limiter.** A fixed-window rate limiter, implemented in the same `app/core/cache.py` module as the cache, guards the lookup-creation endpoint: by default, 10 calls per 60 seconds per user (both values configurable).
3. **Celery broker and result backend.** The `celery_worker` / `celery_beat` services use Redis as both the message broker and the result store for background jobs.

Redis is used across separate **logical database indexes** on the same Redis instance, rather than one shared keyspace:

Redis DB index	Purpose
<code>/0</code>	Cache (provider-result cache and rate-limiter counters, both via <code>app/core/cache.py</code>)
<code>/1</code>	Celery broker (<code>redis://redis:6379/1</code>)
<code>/2</code>	Celery result backend (<code>redis://redis:6379/2</code>)

This separation means a `FLUSHDB` or eviction event against the cache database, for example, cannot collide with in-flight Celery task state, and vice versa -- they are isolated by index even though they share one Redis process. No other use of Redis was found in the

codebase.

It's worth noting that the synchronous SSE lookup pipeline (`POST /api/v1/lookup/stream`) does **not** go through Celery at all -- it runs providers directly and streams results back over the same request. The only scheduled Celery job in the system is an hourly OSINT re-crawl (`run_osint_crawl`) for IOCs looked up in the last 24 hours; Celery's broker/result-backend role is scoped to that background job, not to the interactive lookup path.

Component overview

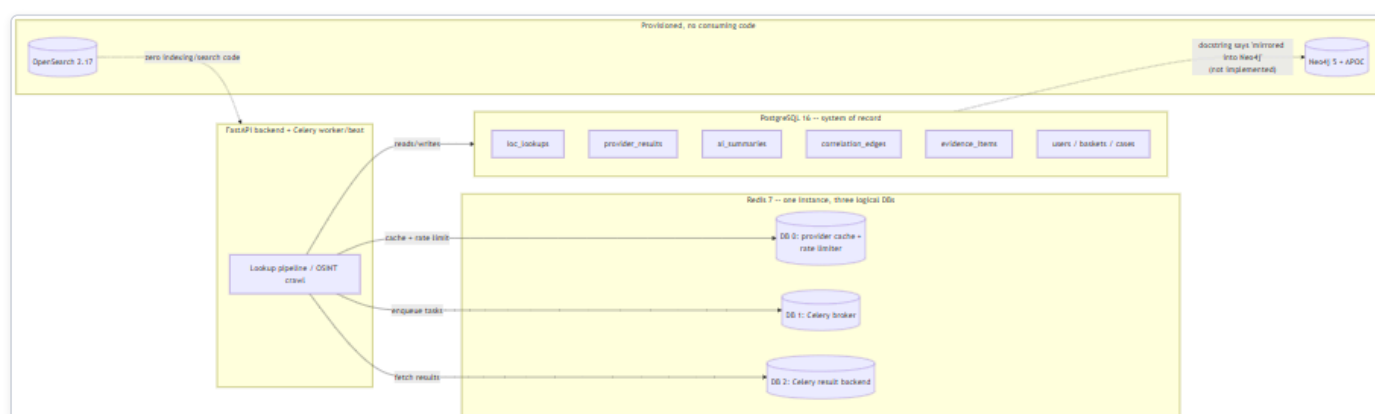


Figure 6 — Diagram: Component overview

Summary

Postgres is the single source of truth for every entity in the platform, including the correlation graph. Redis does three concrete jobs -- provider-result caching, rate limiting, and Celery transport -- cleanly separated by logical database index on one instance. Neo4j and OpenSearch run as containers and appear in configuration, but as of this writing carry no application code that reads or writes to them; they should be read as forward-provisioned infrastructure, not as active components of the current architecture.

Windows Deployment Architecture

This section documents the Windows installer for HORIZON GRID: an Inno Setup package (a Windows installer-authoring tool that compiles a self-extracting setup executable) plus a WinForms configuration wizard (.NET's native desktop UI framework — this is not an Electron or browser-based installer). Together they package the same Docker Compose-based runtime described elsewhere in this document for a single-machine Windows install; they do not reimplement the platform as a native Windows service. Docker Compose remains the actual runtime underneath the installer, per an explicit header comment in `windows/installer.iss`.

What the Installer Actually Does

`installer.iss` copies the following into `%ProgramFiles%\IOC Intelligence Platform\app\` (Inno Setup's `{autopf}` constant):

- `backend/`
- `frontend/`
- `docker-compose*.yaml` (including the production overlay, `docker-compose.prod.yaml`, which strips the development bind-mounts and switches the backend/frontend containers to production start commands)
- `docs/`
- `README.md`
- the `windows/scripts` and `windows/wizard` PowerShell used for post-install configuration and day-to-day operation

Architecture restriction. The installer sets `ArchitecturesAllowed=x64compatible`, restricting installation to 64-bit-compatible Windows systems. This is consistent with — and enforced a second time by — the prerequisite checker's own verification of a 64-bit Windows 10-or-later host (below): the same constraint is checked once at install-package level and once again at runtime. The source material does not document any further stated engineering rationale (e.g. no claim is made here about 32-bit or ARM64 support one way or the other beyond this).

Prerequisite Checks

Before any files are copied, a `[Code]`-section step in `installer.iss` runs `Check-Prerequisites.ps1`, which verifies:

- 64-bit Windows 10 or later
- Administrator privileges
- At least 8 GB RAM (soft check)
- Sufficient free disk space
- Docker Desktop installed and running
- Docker Compose v2 available
- The ports the platform needs are free

Failures surface a "Continue anyway?" prompt rather than a silent abort — the check is advisory-with-friction for at least some of these conditions, not a hard, unbyypassable gate.

File Layout: Program Files vs. ProgramData

The installer follows the standard Windows split between read-mostly program binaries and writable per-machine application data (`windows/scripts/Common.ps1` , lines 28-40):

Location	Purpose	Contents
<code>%ProgramFiles%\IOC Intelligence Platform\app\</code>	Installed application code	<code>backend/</code> , <code>frontend/</code> , compose files, docs, wizard/operational scripts
<code>%ProgramData%\IOC Intelligence Platform\config\.env</code>	Real runtime secrets and configuration	The generated <code>.env</code> consumed by Docker Compose
<code>%ProgramData%\IOC Intelligence Platform\logs\setup.log</code>	Install/setup logging	Setup Wizard log output
<code>%ProgramData%\IOC Intelligence Platform\backups\</code>	Database backups	<code>pg_dump</code> snapshots; the 10 most recent are retained

ACL Protection

`Initialize-DataDirectories` locks the entire `ProgramData\IOC Intelligence Platform\` tree to the Administrators and SYSTEM accounts only, via `icacls` (Windows's command-line ACL — access control list — management tool). This is the only NTFS permission narrowing described in the source material; no per-subfolder differentiation is mentioned beyond this single restrictive ACL applied to the whole tree.

There is one additional wrinkle worth noting precisely because it affects where secrets end up on disk: `docker-compose.yml` 's `env_file: .env` directive resolves relative to the Compose *project* directory, not to whatever path a `--env-file` flag might point at. Because Compose is invoked from inside `{app}\app\` , a helper (`Sync-ComposeEnvFile`) copies the real `.env` from `ProgramData\...\config\.env` into `{app}\app\.env` on every single Compose invocation, and ****re-applies the same Administrators+SYSTEM-only ACL to that copy**** each time. In other words, there are effectively two on-disk `.env` files with secrets in them by design (the `ProgramData` original and the `app-directory` working copy Compose actually reads), and both are kept under the restrictive ACL rather than only the first one.

Secrets themselves are generated using a CSPRNG (cryptographically secure pseudo-random number generator) — specifically .NET's `RandomNumberGenerator` via a `New-RandomSecret` helper, not PowerShell's `Get-Random` — and written out by `Write-EnvFile.ps1` .

Setup Wizard Page Flow

`Setup-Wizard.ps1` is a WinForms application. On launch it self-elevates via UAC (User Account Control) if it isn't already running with a real, non-filtered elevated token. It then walks the operator through a fixed sequence of pages:

Welcome → Admin Account → AI Configuration → Provider Configuration → Port Review → Summary/Install → Finish

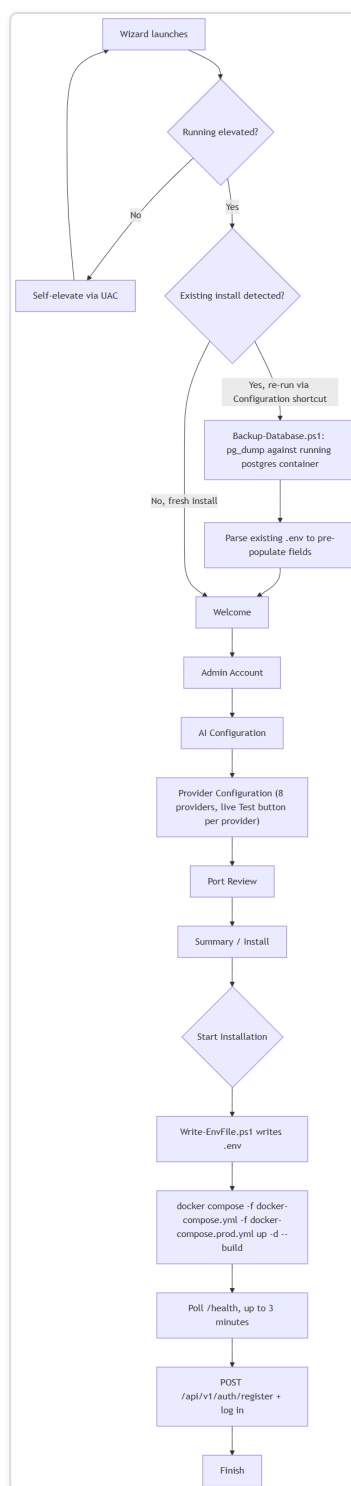


Figure 7 — Diagram: Setup Wizard Page Flow

Notes on individual pages:

- **Admin Account** — collects the credentials for the operator account the installer will register once the stack is up. Per the platform's own registration logic, the *first* account ever registered against a fresh database is automatically granted the admin role, with no manual database edit required; every registration attempt after that is rejected with **403 Forbidden** rather than silently

creating a lesser account. That bootstrap behavior is what makes it safe for this page to simply be "create the admin account" with no separate role picker.

- **Provider Configuration** — presents exactly 8 of the platform's 18 registered intelligence providers, each with a live "Test" button that calls `POST /api/v1/providers/{id}/test` against the running backend: VirusTotal, AbuseIPDB, OTX, the combined abuse.ch group (URLhaus/ThreatFox/MalwareBazaar — one free Auth-Key covers all three), NVD, Hybrid Analysis, Censys, and PhishTank. The wizard's own inline notes match the backend's behavior exactly for the cases checked: Censys requires both a Personal Access Token and an Organization ID, the abuse.ch key is shared across three connectors, and NVD works without a key at a lower rate limit. The remaining 10 backend providers are deliberately absent from this page for two different reasons: 6 (Certificate Transparency lookups, CISA KEV, MITRE ATT&CK, WHOIS/RDAP, Spamhaus, and the internal OSINT crawler) require no credential to collect at all, so there is nothing for this page to configure and no corresponding test handler exists for them either; the other 2 (urlscan.io, Google Safe Browsing) **do** require a credential but are still absent from the wizard on both platforms identically — both are configured after install from the app's own Providers page instead.
- **Port Review** — lets the operator confirm/adjust the host ports the stack will bind, following on from the prerequisite check's "ports free" verification.

What "Start Installation" Actually Executes

Clicking "Start Installation" on the Summary/Install page runs, in order:

1. Writes `.env` via `Write-EnvFile.ps1`, using the values collected across the previous pages plus CSPRNG-generated secrets.
2. Runs `docker compose -f docker-compose.yml -f docker-compose.prod.yml up -d --build` — the production overlay swaps in production start commands and drops the development bind-mounts.
3. Polls the backend's `/health` endpoint for up to 3 minutes, waiting for the stack to come up.
4. Registers the admin account by calling `POST /api/v1/auth/register` with the credentials from the Admin Account page, then logs in.

Nothing here is a distinct "installer-native" install step beyond orchestrating the same Docker Compose commands and HTTP calls an operator could run by hand — the wizard's value is sequencing and validating them, not replacing them.

Start Menu Shortcuts

The installer's `[Icons]` section creates the following (plus an optional desktop icon):

Shortcut	What it does
Open Platform	Runs <code>Open-Platform.ps1</code> (also the default desktop icon action): checks stack health first, starts Docker Desktop if it isn't already running, starts the stack, then opens the browser to the platform.
Configuration	Re-runs the Setup Wizard. On an existing install this sets an upgrade flag (<code>\$State.IsUpgrade</code>) that changes the wizard's behavior — see Upgrade/Reconfigure below.
Start Platform	Runs <code>Service-Start.ps1</code> : <code>docker compose up -d</code> , then polls <code>/health</code> .
Stop Platform	Runs <code>Service-Stop.ps1</code> : <code>docker compose stop</code> (no <code>-v</code> flag, so container data/volumes are preserved, not deleted).
Restart Platform	Listed in the installer's icon set; the facts available do not detail its script-level implementation beyond the name (presumably a stop-then-start sequence, not confirmed).
Service Status	Listed in the installer's icon set; specific implementation not detailed in the source material beyond the name.
Backup Database Now	Manually triggers an on-demand database backup using the same <code>pg_dump</code> -against-the-running-postgres-container mechanism (<code>Backup-Database.ps1</code>) that runs automatically before an upgrade (see below).
Diagnostics	Listed in the installer's icon set; specific implementation not detailed in the source material beyond the name.
Documentation	Opens the installed documentation (<code>docs/</code> , <code>README.md</code>) copied in at install time.
Uninstall	Invoked via <code>[Code]</code> in <code>installer.iss</code> ; offers "Remove Application" (keeps <code>ProgramData</code> and Docker volumes intact) versus "Remove Everything" (requires typing <code>DELETE</code> to confirm; runs <code>docker compose down -v</code> and deletes <code>ProgramData</code>).

Upgrade / Reconfigure Flow

Re-running the Setup Wizard from the **Configuration** shortcut on a machine that already has the platform installed sets `$State.IsUpgrade = true` internally, which changes two things before the operator sees any page:

1. **Pre-population:** the wizard parses the existing `.env` (from `ProgramData\...\config\.env`) and uses it to pre-fill the wizard's fields, so re-running Configuration is a "review and adjust" flow rather than starting from a blank slate.
2. **Pre-install backup:** before anything else changes, the wizard backs up the database by running `Backup-Database.ps1` , which executes `pg_dump` inside the already-running Postgres container. This backup lands in `ProgramData\...\backups\` , alongside the same rolling most-recent-10-retained snapshots used elsewhere.

From there the wizard proceeds through the same page flow (Welcome → Admin Account → ... → Summary/Install → Finish) and "Start Installation" re-runs the same `docker compose ... up -d --build / health-poll` sequence described above. The source material does not specify whether the final admin-account registration call is skipped or altered on an upgrade path versus a fresh install — this detail is **not confirmed** and should not be assumed either way without checking the wizard script directly.

Summary

The Windows installer's job is narrowly scoped: verify the host can run the stack, lay down application files under Program Files, collect configuration through a WinForms wizard, generate and lock down secrets under a restrictive ACL in ProgramData, and drive the same `docker compose` commands and REST calls an operator could otherwise run manually. It intentionally leaves Docker Compose as the real runtime rather than replacing it with a native Windows service, and it protects the resulting secrets primarily through NTFS ACLs (Administrators + SYSTEM only) on the `.env` file(s) and CSPRNG-based secret generation, rather than through any OS-level secret store.

Security Architecture

This appendix section documents the authentication, authorization, secrets-handling, network-exposure, rate-limiting, and CORS behavior of the HORIZON GRID backend (the FastAPI service that receives IOC — Indicator of Compromise — lookup requests from the web client). All claims below are traceable to source-code inspection of the platform's backend and Windows installer; anything not directly confirmed in code is marked "Not confirmed" rather than estimated.

Authentication

The backend issues JSON Web Tokens (JWT) using `python-jose` with the HS256 signing algorithm (`backend/app/auth/security.py`). Two token types are issued on login:

Token	Lifetime
Access token	30 minutes
Refresh token	7 days

Password storage uses `passlib` with `bcrypt` hashing — plaintext passwords are never persisted; only the bcrypt hash is stored on the `users` table.

First-user-becomes-admin bootstrap

`POST /auth/register` contains a specific bootstrap rule: **the first user ever registered on a given instance is automatically granted the `admin` role; every registration attempt after that is rejected with `403 Forbidden`** ("Self-registration is closed. Ask an administrator to create your account from the Administration page.") rather than being granted a lesser role. No manual database edit is required to stand up an initial administrator account — this is also how the Windows Setup Wizard provisions the admin account at the end of installation (it calls this same registration endpoint). Every account after that first admin must be created by an existing admin, via `POST /api/v1/admin/users` or the `/admin` frontend page, which lets the admin choose the new account's role up front. `docs/SECURITY.md` describes this identical mechanism.

Role-Based Access Control (RBAC)

The platform has exactly **three roles**: `admin` , `analyst` , and `viewer` . Permissions are modeled as a permission-string matrix (`ROLE_PERMISSIONS` in `app/models/user.py`) and enforced per-route by a `require_permission()` dependency (`app/auth/rbac.py`) — a route either declares a required permission string or it doesn't; there is no implicit fallback check.

Documentation correction made during this review: `docs/SECURITY.md` states that no route enforces the `provider:manage` permission. That claim is stale. The provider-credential test endpoint, `POST /api/v1/providers/{provider_id}/test` (`backend/app/api/routes/providers.py` , line 24), does call `require_permission("provider:manage")` — confirmed directly in the route code. This endpoint is what the Setup Wizard's per-provider "Test" button invokes, so the permission gate is exercised as part of the normal install flow, not a dead code path.

Two other admin-tier permissions defined in the same matrix were checked. One remains unenforced; the other, `audit:read` , is now enforced as of the runtime provider/AI configuration feature described under Secrets Management below:

Permission	Status
<code>provider:manage</code>	Enforced — gates <code>POST /api/v1/providers/{provider_id}/test</code> and the new <code>/api/v1/runtime/*</code> provider- and AI-configuration endpoints
<code>user:manage</code>	Defined in the RBAC matrix; no route currently calls <code>require_permission("user:manage")</code>
<code>audit:read</code>	Enforced — gates <code>GET /api/v1/runtime/audit-log</code>

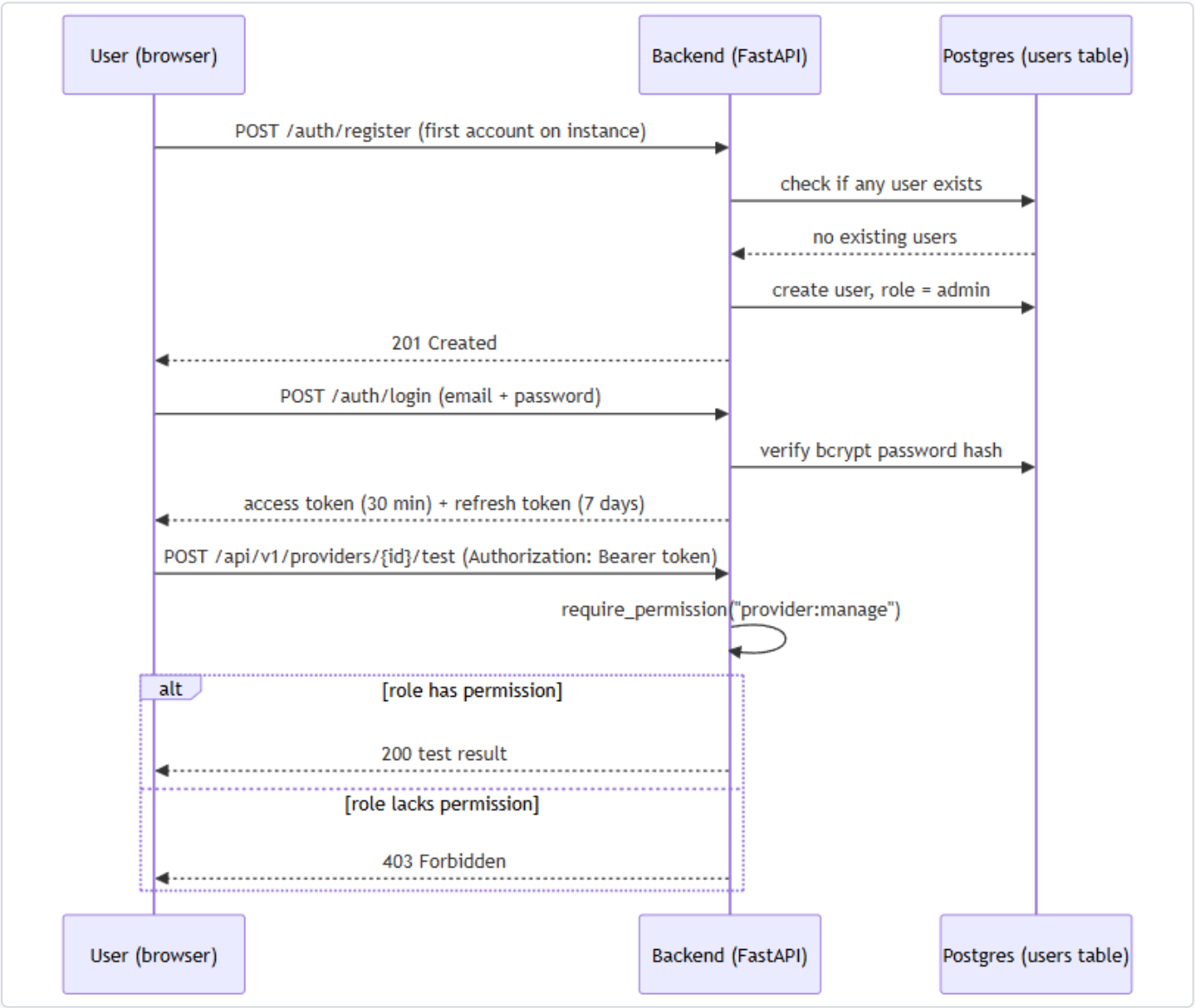


Figure 8 — Diagram: Role-Based Access Control (RBAC)

Secrets Management

Runtime configuration, including secrets, is loaded from a `.env` file via `pydantic-settings`. `.env` itself is git-ignored; only a `.env.example` template is tracked in source control.

One default value is a genuine risk if left unchanged: `jwt_secret_key` falls back to the literal string `"change-me-in-production"` when no value is set (`backend/app/core/config.py`, line 23). Any deployment that ships with this default unedited would have a publicly-known JWT signing secret.

On Windows installs, the Setup Wizard avoids this by generating secrets itself rather than relying on the code default: it uses a cryptographically secure random number generator (`New-RandomSecret`, backed by .NET's `RandomNumberGenerator` — explicitly *not* PowerShell's `Get-Random`, which is not cryptographically secure) and writes the result via `Write-EnvFile.ps1`. The resulting `.env` file's filesystem permissions are then locked down to the Administrators and SYSTEM accounts only, via `icacls`.

Setup Wizard credential-testing reliability fix

A review of the Setup Wizard's per-credential "Test Connection" buttons — used to validate both third-party provider API keys and AI-backend credentials (Ollama, Anthropic, Bedrock, Gemini, Groq) before installation is finalized — found two distinct root causes for unreliable behavior in this flow, both corrected during this review.

Session-acquisition bug. Every Test Connection button's request depends on an authenticated session token (`$State.AccessToken`). That token was previously set in exactly one place in the wizard: deep inside the "Start Installation" button's click handler on the final Summary page. As a result, no credential could be tested until installation had already been fully completed. On a reconfigure run in particular — where an administrator deliberately leaves the Admin Account page blank in order to keep an existing account unchanged — no login request was issued at all, so every Test Connection click failed with a "create the admin account first" message even though a working admin account and platform were already running. The fix is a new function, `Get-OrCreateWizardSession` (`windows/wizard/Setup-Wizard.ps1`), invoked on demand directly from each Test Connection click handler. It performs a login only, using whatever email and password are currently typed into the Admin Account page fields, and never registers or modifies an account.

.env serialization hardening. A second, independent bug existed in the writer responsible for persisting configuration values to disk, `windows/scripts/Write-EnvFile.ps1`: values were written without any escaping. A `#` character occurring anywhere inside a password or API key was interpreted by the `.env` parser as the start of a comment, silently truncating the remainder of that line, and an embedded newline in a value could similarly corrupt the file's line structure. This is fixed with a new `ConvertTo-SafeEnvValue` helper, applied to every value before it is written.

Neither issue involved the JWT signing or RBAC mechanisms described elsewhere in this document; both were specific to the Windows installer's wizard-to-backend session handling and its own configuration-file writer.

Encryption at rest for runtime-configured provider and AI credentials

A newer capability lets an administrator add, configure, enable/disable, and switch AI-backend and IOC-provider credentials at runtime through a Manage Providers UI, without editing `.env` or restarting the containers. Because this moves credential entry out of the git-ignored `.env` file and into the database, those values are encrypted before they are ever written.

Credentials submitted through this flow are encrypted with Fernet symmetric encryption (`backend/app/core/crypto.py`) and persisted as ciphertext in the `provider_runtime_configs` table's `encrypted_credentials` column — the database never holds a plaintext runtime-configured credential. This is not merely an inference from reading the code: a direct SQL query against the running database showed only Fernet-format tokens (the `gAAAAAB...` prefix), never a real key value.

The Fernet key used for this encryption is resolved one of two ways: an explicit, separately-configured `encryption_master_key` setting, if the operator has set one, or — by default — a key deterministically derived via HKDF from the platform's existing `jwt_secret_key`. The HKDF fallback means every existing installation already has a usable encryption key with no migration step required, but it is worth stating plainly what this tradeoff is: because the fallback key is derived from `jwt_secret_key` rather than generated and stored independently, it is defense-in-depth against exposure of the raw database, not HSM-grade key separation — compromise of `jwt_secret_key` would also expose the derived encryption key. An operator wanting stronger separation should set `encryption_master_key` explicitly.

Every configure, enable/disable, activate, and test-connection action taken against a provider or AI backend through this flow is separately recorded in a new `config_audit_log` table (timestamp, actor email, action, and a human-readable `detail` field). The write path for that table only ever accepts descriptive text as `detail` — it is not a place a credential value can be written. This was checked directly, not just by reading the code: after exercising the full configure/enable/disable/test/activate flow, a grep of the audit log table's contents and of the backend logs found no real secret value in either.

Network Exposure

Every datastore container's port mapping in `docker-compose.yml` — Postgres, Redis, both Neo4j ports, and OpenSearch — uses the explicit `127.0.0.1:<host_port>:<container_port>` binding form, which restricts that port to the loopback interface only (not reachable from other machines on the network as ordinarily configured).

The `backend` and `frontend` containers are published differently: their compose entries use the shorthand form (e.g. `"${HOST_PORT_BACKEND:-8000}:8000"`) with no explicit host-IP prefix. Docker's shorthand publishing binds to all host network interfaces by default — so, unlike the datastores, the backend API and frontend web app are reachable from other machines on the local network unless something else (a firewall rule, router configuration, etc.) restricts that.

Note on rigor: `docker-compose.yml` contains first-person comments asserting that live testing confirmed specific network-reachability behavior from another LAN machine. This section does not rely on that narrative comment as evidence — only the literal, machine-checkable YAML binding syntax described above was treated as verified fact.

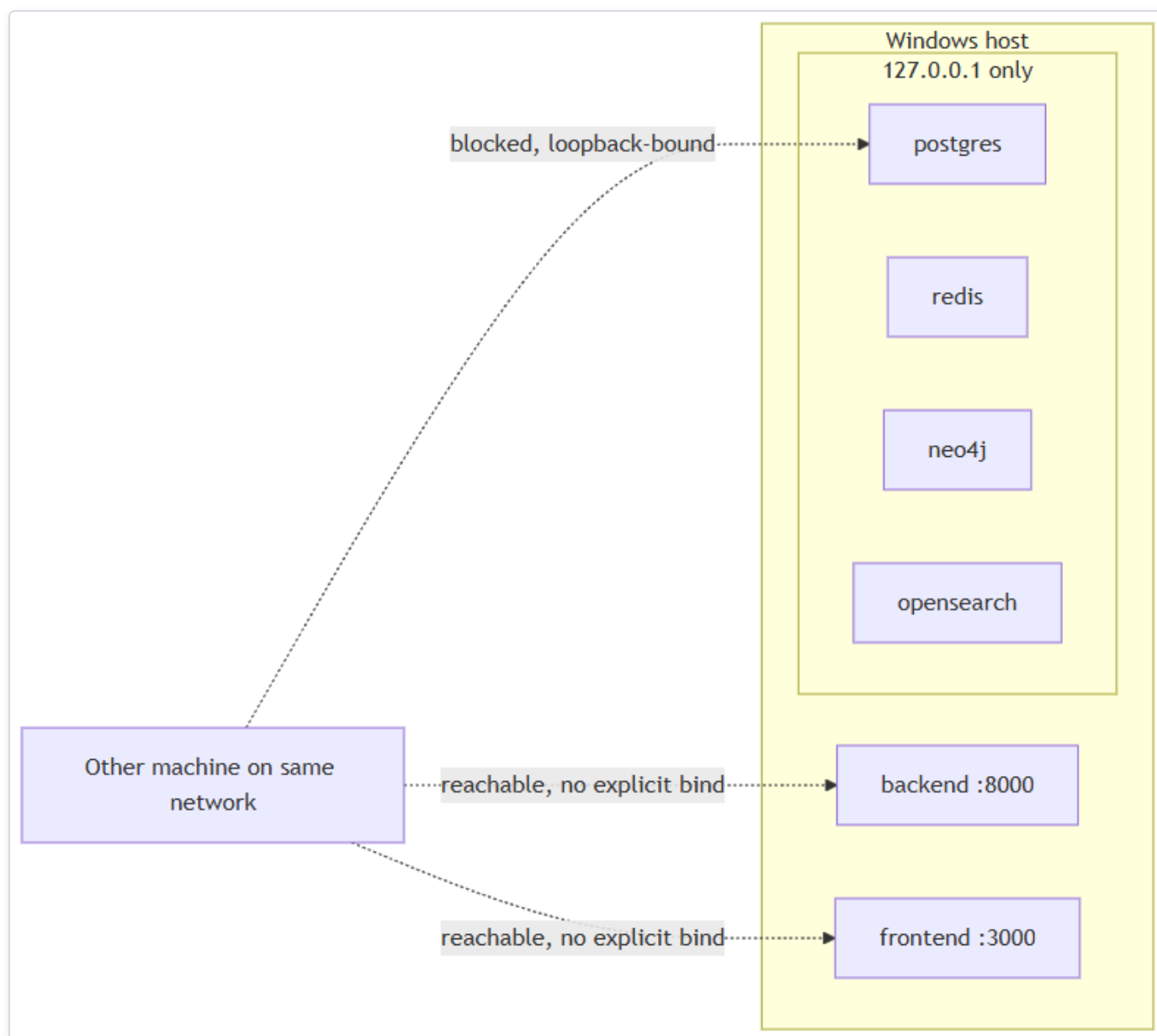


Figure 9 — Diagram: Network Exposure

Rate Limiting

The codebase implements exactly **one** rate limiter: a Redis fixed-window limiter that applies solely to `POST /lookup/stream` (the IOC lookup endpoint), defaulting to 10 calls per 60 seconds per authenticated user — both figures are configurable.

`/auth/login` has a second limiter, added in a later mission-critical-reliability review (v0.2.3): a per-account Redis fixed-window limiter keyed by email (`login:{email}`, not source IP), defaulting to 10 attempts per 60 seconds, both configurable, returning `429` once exceeded. `/auth/register` still has no rate limiting of any kind.

CORS Behavior

Cross-Origin Resource Sharing is configured in `backend/app/main.py` (lines 27-33) and its behavior depends entirely on the `debug` setting:

- When `settings.debug` is `True` (the default), `allow_origins` is set to `["http://localhost:3000"]`.
- When `debug` is `False`, `allow_origins` is set to `[]` — an empty list, meaning **zero** cross-origin browser requests are permitted.

There is no `CORS_ORIGINS` environment variable or other configuration knob to adjust this. Practically, this means a `DEBUG=false` production deployment cannot serve a browser frontend from any other origin than the backend itself without a source-code change.

Other Input-Handling Notes

Two related facts are worth surfacing here because they feed directly into the hardening list below:

- `LookupCreateRequest.value` (the raw IOC string submitted for lookup) is an unconstrained `str` on the server side. Classification into an IOC type happens via regex-based heuristics (`app/ioc/detector.py`), which is pattern classification, not input sanitization.
- `RegisterRequest.password` carries a server-side 8-character minimum (`Field(min_length=8, ...)`, `app/schemas/auth.py`), enforced by Pydantic for every account, not just the bootstrap admin — but no complexity rule (digit/symbol/case) beyond that length floor. The Setup Wizard's own UI also enforces an 8-character minimum when creating the bootstrap admin account, matching the backend constraint rather than substituting for it.

Areas for Future Hardening

Based only on the facts established above:

- **Default JWT secret is a known literal.** `jwt_secret_key` defaults to `"change-me-in-production"` if the operator never sets it explicitly — any instance that ships with this unedited has a predictable signing key.
- **No rate limiting on `/auth/register`.** `/auth/login` gained a real per-account rate limiter in v0.2.3 (see Rate Limiting above); registration has no throttling against repeated attempts.
- **No server-side password complexity rule beyond length.** The backend enforces an 8-character minimum for every account (`app/schemas/auth.py`), but no digit/symbol/case requirement on top of that.
- **One admin-tier permission still has no enforcing route.** `user:manage` is defined in the RBAC permission matrix but, unlike `provider:manage` and `audit:read`, is not currently checked by any route.
- **Backend and frontend ports are not loopback-restricted by default.** Datastores (Postgres, Redis, Neo4j, OpenSearch) are bound to `127.0.0.1` only; the backend API and frontend web app publish on all host interfaces by default, unless the operator adds their own network restriction.
- **Runtime-credential encryption key defaults to a derived, not independently-generated, secret.** Unless an operator explicitly sets `encryption_master_key`, the Fernet key protecting runtime-configured provider/AI credentials (see Secrets Management above) is deterministically derived via HKDF from `jwt_secret_key` rather than generated and stored separately. That is a reasonable defense-in-depth default — it means every existing install already has a working encryption key — but it is not equivalent to HSM-grade key separation: compromise of `jwt_secret_key` would also expose the derived encryption key.

Linux Packaging Architecture

Application layer: unchanged

HORIZON GRID's application layer is identical on Linux and Windows. Both platforms run the same Docker Compose stack -- Postgres, Redis, Neo4j, OpenSearch, the FastAPI backend, the Next.js frontend, and two Celery workers -- built from the same `backend/`, `frontend/`, `docker-compose.yml`, and `docker-compose.prod.yml`. Neither platform bundles Node/pip dependencies or pre-built images in its installer artifact; both install them the same way, inside the containers, on the first `docker compose up --build`. Both platforms' Compose project directory shares the basename `app`, so both produce the identical Docker Compose project label (`com.docker.compose.project=app`). There is no architectural divergence at this layer -- it is the same code, the same images, the same runtime behavior.

Installer/lifecycle layer: the only difference

All Linux-specific work is confined to the outer packaging and lifecycle-management layer, which replaces the Windows installer stack component-for-component:

Windows	Linux
Inno Setup installer	<code>.deb</code> package (<code>horizon-grid_0.2.0_amd64.deb</code>)
WinForms setup wizard	Python CLI setup wizard (terminal-based, <code>horizon-grid configure</code>)
Windows service registration	systemd unit (<code>horizon-grid.service</code>)
Start Menu shortcut group	<code>horizon-grid</code> CLI endpoint + desktop menu launcher
ProgramData config (ACL-locked)	<code>/etc/horizon-grid/.env</code> (root:root, chmod 600)

The CLI wizard (`linux/wizard/setup_wizard.py`) follows the same flow and issues the same HTTP calls as its Windows counterpart (`windows/wizard/Setup-Wizard.ps1`): Welcome -> Administrator Account -> AI Configuration -> Threat Intelligence Providers -> Network Ports -> Summary -> write `.env` -> `docker compose up -d --build` -> wait for `/health` -> register the admin account -> confirm login. The systemd unit is registered at install time but never auto-started -- nothing runs until the admin completes the wizard, matching the Windows installer's own "nothing auto-starts before configuration" behavior.

Installed file layout (FHS)

Path	Purpose	Windows analog
/opt/horizon-grid/app/	Application code (backend/, frontend/, compose files, docs)	Program Files
/etc/horizon-grid/.env	Config/secrets, root:root, chmod 600	ProgramData config (ACL-locked)
/var/lib/horizon-grid/backups/	pg_dump snapshots	ProgramData backups
/var/log/horizon-grid/setup.log	Setup/runtime log	ProgramData logs\setup.log
/usr/bin/horizon-grid	CLI entrypoint	--
/lib/systemd/system/horizon-grid.service	systemd unit	Windows service registration
/usr/share/applications/horizon-grid.desktop	Desktop menu launcher	Start Menu shortcut

Why no AppImage

An AppImage packages a single GUI executable for double-click launch. HORIZON GRID is a multi-container Docker Compose platform accessed via a browser, not a single GUI binary -- there is no one executable to wrap. The `.deb` + systemd + CLI-wizard combination is the direct Linux analog of what the Windows installer already does: neither platform reimplements the application as a native, self-contained desktop program. Docker Compose is the real runtime on both.